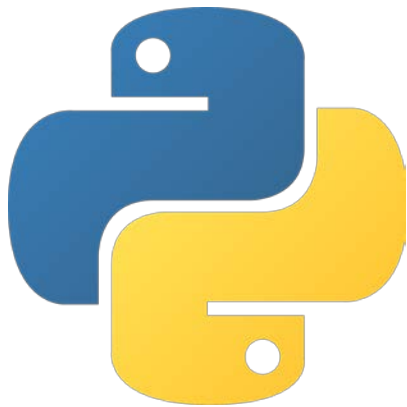




Introducción a Python y su Aplicación al Trading

Alberto Muñoz
X-Trader.net

14 de febrero de 2018



¡¡¡Hoy NO hay Bolas
de Dragón!!!

ÍNDICE

1. ¿QUÉ ES PYTHON?
2. PAQUETES PARA TRADERS EN PYTHON
3. PRIMEROS PASOS EN PYTHON
4. MI PRIMER SISTEMA DE TRADING EN PYTHON
5. MACHINE LEARNING EN PYTHON
6. RECURSOS PARA TRADERS EN PYTHON



¿QUÉ ES PYTHON?

¿QUÉ ES PYTHON?

- Python es un lenguaje interpretado, dinámico y orientado a objetos creado a finales de los ochenta por Guido van Rossum del Centrum Wiskunde & Informatica (CWI) holandés. El nombre de este lenguaje proviene de la afición de su creador por los humoristas británicos Monty Python.
- En 1995, van Rossum se muda a la Corporation for National Research Initiatives (CNRI) en EEUU desde donde impulsó la iniciativa Computer Programming for Everybody (CP4E), con el fin de hacer la programación más accesible a más gente y aumentar el nivel de 'alfabetización' básico en lenguajes de programación.
- En el año 2001, van Rossum fue premiado con el FSF Award for the Advancement of Free Software.

¿QUÉ ES PYTHON?

LOS 12 MANDAMIENTOS DE PYTHON

1. Los nombres de los módulos deben estar en minúsculas – `hola.py`
2. Los nombres de las clases deben usar CamelCase
3. Los métodos y funciones deben usar minúsculas_con_guion_bajo
4. Los métodos privados para uso interno comienzan con `_guion_bajo`
5. Los atributos de clase con `__doble_guion_bajo`
6. Las constantes en el primer nivel del código (las que no se encuentran dentro de una función o una clase) deben usar LETRAS MAYÚSCULAS. Usar demasiadas constantes puede hacer que tu código sea menos reutilizable.

¿QUÉ ES PYTHON?

LOS 12 MANDAMIENTOS DE PYTHON

7. Si una variable en una función o método es tan temporal que no puedes darle un nombre, utiliza i para la primera, j para la segunda, y k para la tercera.
8. Indenta con cuatro espacios por nivel. Sin tabuladores. Si rompes este mandamiento serás lapidado en la plaza del pueblo.
9. Las líneas no deberían tener nunca más de 80 caracteres. Divide las líneas usando una barra invertida. No necesitas hacer esto si hay paréntesis, llaves o corchetes.
10. Espacio después de una coma (huevos, con, jamon)

¿QUÉ ES PYTHON?

LOS 12 MANDAMIENTOS DE PYTHON

11. Espacio antes y después de un operador $i = i + 1$

12. Escribe cadenas de documentación para todos los módulos, funciones, clases y métodos públicos. Python es una comunidad internacional, así que utiliza el inglés para las cadenas de documentación, los nombres de los objetos y los comentarios

¿QUÉ ES PYTHON?

- Actualmente podemos encontrar más de 128.000 *packages* o paquetes que permiten ampliar enormemente sus funcionalidades. Podéis ver la lista completa en PyPI: <https://pypi.python.org/pypi>
- En ese listado podemos encontrar todo tipo de herramientas que abarcan campos tales como bases de datos, juegos, telecomunicaciones, estadística, ingeniería o procesamiento de textos.
- Python puede descargarse desde <https://www.python.org/downloads/>. Está disponible para Windows, Linux, Mac OS X o incluso para dispositivos como un teléfono móvil.
- Existen dos versiones: 2.7.14 y 3.6.4. Por razones de compatibilidad, para los ejemplos utilizaremos la versión 2.7.14.

¿QUÉ ES PYTHON?

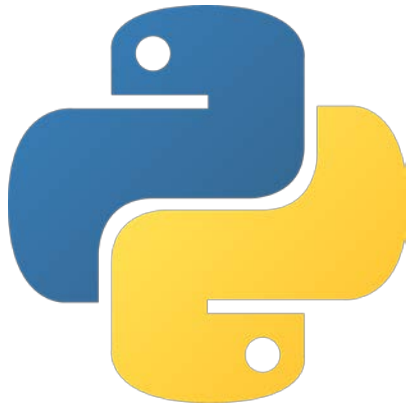
VENTAJAS

- Amplísima colección de librerías y paquetes (más del doble que R). Podemos hacer virtualmente casi cualquier cosa.
- Es gratuito y multiplataforma.
- Se integra con otras plataformas y lenguajes (Visual Studio .NET, SPSS, OpenOffice, Gimp, Java)
- La representación gráfica ha mejorado bastante con librerías como Seaborn o Plotly.

¿QUÉ ES PYTHON?

DESVENTAJAS

- Al igual que R, como lenguaje de ejecución en tiempo real es lento (motivo por el cual se prefiere C++/Java/C# para la ejecución de órdenes) al no estar compilado sino interpretado.
- No es la mejor opción para programación multi-hilo.

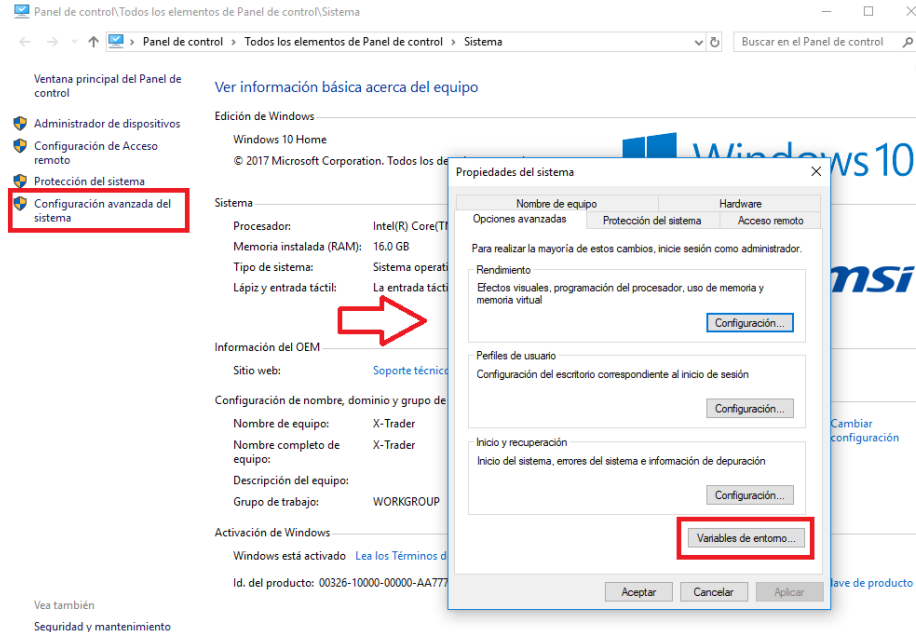


PAQUETES PARA TRADERS EN PYTHON

PAQUETES PARA TRADERS EN PYTHON

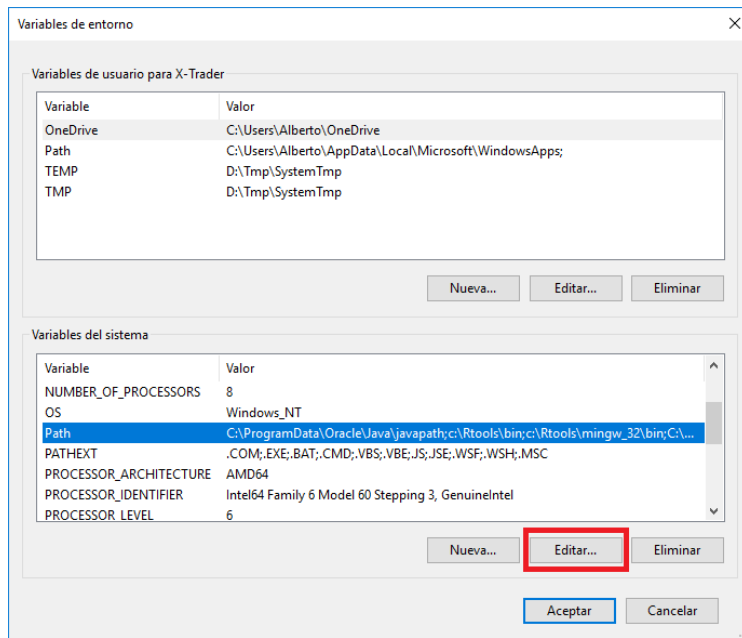
Primero el Path del Sistema

- Lo primero de todo: ¡¡¡aseguraos de que tenéis bien el PATH del sistema!!!



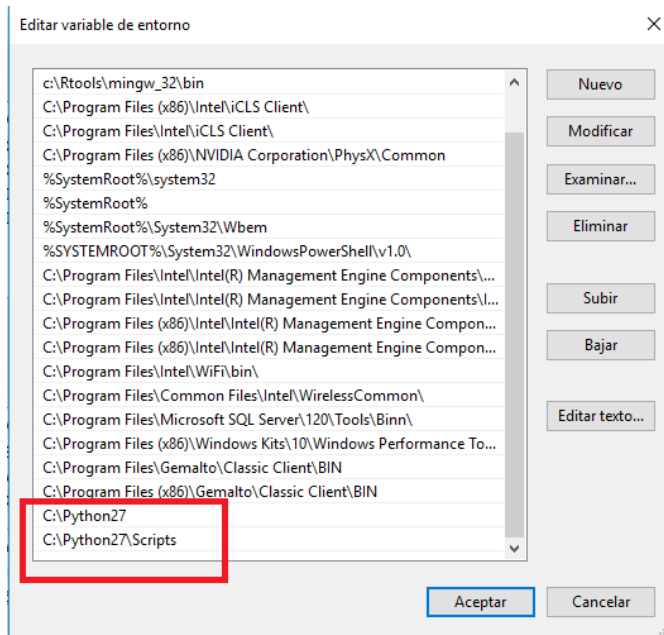
PAQUETES PARA TRADERS EN PYTHON

Primero el Path del Sistema



PAQUETES PARA TRADERS EN PYTHON

Primero el Path del Sistema



PAQUETES PARA TRADERS EN PYTHON

Instalar Paquetes en Python

- Desde el Símbolo de Sistema verificamos que tenemos Pip instalado:

```
python -m ensurepip --default-pip
```

- Es recomendable añadir setuptools y wheel:

```
pip install --upgrade pip setuptools wheel
```

- También interesa instalar Microsoft Visual C++ Compiler for Python 2.7 por si os lo pidiera algún paquete:

<https://www.microsoft.com/en-gb/download/details.aspx?id=44266>

PAQUETES PARA TRADERS EN PYTHON

Cómo Instalar Paquetes en Python

- Para instalar paquetes vamos al Símbolo del Sistema y escribimos:

```
pip install <nombre_del_paquete>
```

PAQUETES PARA TRADERS EN PYTHON

DESCARGA DE DATOS

- **Pandas-DataReader:** <https://pypi.python.org/pypi/pandas-datareader/>
Permite descargar datos de, entre otros, Google Finance, Morningstar, Quandl, FRED, World Bank, OCDE, Eurostat, etc.
- **Quandl:** <https://pypi.python.org/pypi/Quandl>
- **Alpha Vantage:** https://pypi.python.org/pypi/alpha_vantage/0.1.6
- **Tiingo:** <https://pypi.python.org/pypi/tiingo>
Cliente REST para la API de la plataforma de datos Tiingo.
- **Barchart on Demand:** <https://pypi.python.org/pypi/barchart-ondemand-client-python/0.4.0/>
Permite descargar datos mediante la API BarChartOnDemand

PAQUETES PARA TRADERS EN PYTHON

ANÁLISIS DE DATOS

- **NumPy:** <http://www.numpy.org/>
Gestiones de arrays n-dimensionales, álgebra lineal, transformadas de Fourier, números aleatorios.
- **Pandas:** <http://pandas.pydata.org/>
Series temporales, gestión de arrays, carga de datos en CSV y XLS, funciones estadísticas, etc.
- **SciPy:** <https://www.scipy.org/>
Recopila en un solo paquete NumPy, Pandas, Matplotlib, Sympy y algunas librerías propias. También incluye IPython, un potente shell para Python.
- **Statsmodels:** <http://www.statsmodels.org/dev/index.html>
Todo tipo de modelos y tests estadísticos, bastante completo.

PAQUETES PARA TRADERS EN PYTHON

DISTRIBUCIONES CIENTÍFICAS DE PYTHON

- **Anaconda:** <https://www.anaconda.com/>
- **Canopy:** <https://www.enthought.com/product/canopy/>
- **Python(x,y):** <https://python-xy.github.io/>
- **WinPython:** <https://winpython.github.io/>
- **Pyzo:** <http://www.pyzo.org/>

PAQUETES PARA TRADERS EN PYTHON

GRÁFICOS

- **MatPlotLib:** <https://matplotlib.org/>
La librería gráfica fundamental para Python.
- **SeaBorn:** <https://seaborn.pydata.org/>
Librería gráfica orientada a visualización de datos estadísticos
- **Plotly:** <https://pypi.python.org/pypi/plotly>
Posiblemente la que tiene los mejores gráficos en Python, requiere registro.

PAQUETES PARA TRADERS EN PYTHON

LIBRERÍAS ESPECÍFICAS PARA TRADERS

- **TA-Lib:** <http://ta-lib.org/>
Amplia colección de indicadores, osciladores y patrones candlestick.
- **Trade:** <http://trade.readthedocs.io/>
Permite llevar registros de operaciones.
- **Zipline:** <http://www.zipline.io/>
Excelente para realizar backtests de estrategias, es la base sobre la que funciona Quantopian.
- **Cuemacro:** <http://www.cuemacro.com/>
Se estructura en 3 paquetes: Finmarketpy (backtest de estrategias), Findatapy (descarga de datos) y Chartpy (gráficos de calidad).

PAQUETES PARA TRADERS EN PYTHON

LIBRERÍAS ESPECÍFICAS PARA TRADERS

- **QuantSoftware Toolkit:** <http://wiki.quantsoftware.org/>
Bastante completo, permite realizar backtests, simulaciones y gráficos normalizados. Incluye indicadores y la posibilidad de usar machine learning.
- **Backtrader:** <https://www.backtrader.com/>
Hace casi de todo: desarrollo de estrategias, indicadores, gráficos, backtests, recepción de datos, simulación de órdenes, ejecución multitimeframe simultánea de estrategias en Interactive Brokers, Oanda y Visual Chart.

PAQUETES PARA TRADERS EN PYTHON

CONEXIÓN A BROKERS

- **QTPyLib:** <https://pypi.python.org/pypi/QTPyLib/1.5.77>
Recibe datos y ejecuta estrategias contra la TWS de Interactive Brokers (solo Python 3, para Python 2 tenemos **lbPy2** (<https://pypi.python.org/pypi/lbPy2/>))
- **Conectar a Metatrader mediante ZeroMQ (PyZMQ):**
<https://blog.darwinex.com/zeromq-interface-python-r-metatrader4/>
- **SimpleFIX:** <https://pypi.python.org/pypi/simplefix>

PAQUETES PARA TRADERS EN PYTHON

!!!Y MÁS!!!

- Podéis encontrar una lista muy amplia de librerías en:

<https://github.com/>

<https://pypi.python.org/pypi?:action=browse&c=375&c=385>

<https://github.com/wilsonfreitas/awesome-quant#python>

PAQUETES PARA TRADERS EN PYTHON

ENTORNOS DE DESARROLLO

- **Spyder (Scientific PYthon Development EnviRonment)** (<https://pypi.python.org/pypi/spyder>). Recomendable si venís de Matlab o R. Viene incluido en Anaconda.
- **PyCharm** (<https://www.jetbrains.com/pycharm/>). Ideal si venís de Java y habéis programado con IntelliJ IDEA de JetBrains.
- **Rodeo** (<https://www.yhat.com/products/rodeo>). Perfecto si os gusta RStudio.

PAQUETES PARA TRADERS EN PYTHON

ENTORNOS DE DESARROLLO

- **Atom** (<https://atom.io/>). Desarrollado por la gente de GitHub. Si habéis programado en Ruby on Rails, PHP o Java este IDE os gustará.
- **Jupyter Notebook** (<http://jupyter.org/>). Desarrollado en 2014 a partir de IPython, permite crear “notebooks” educativos. La ejecución de scripts se realiza directamente desde el navegador. Se puede probar online en <https://try.jupyter.org/>. Si queréis ejecutarlo en vuestro ordenador, instalad el paquete y teclead “jupyter notebook” directamente desde el Símbolo de Sistema.

PAQUETES PARA TRADERS EN PYTHON

The screenshot shows the JupyterLab interface. At the top, there's a navigation bar with a Jupyter logo, a 'Logout' button, and a search bar. Below the navigation bar, there's a 'Select items to perform actions on them.' section with buttons for 'Files', 'Running', and 'Clusters'. To the right of this section are buttons for 'Upload', 'New' (highlighted with a red box), and a refresh icon. The main area is divided into two panes. The left pane shows a file browser with a tree view of the file system, including folders like 'DLLs', 'Doc', 'etc', 'include', 'Lib', 'libs', 'Scripts', 'share', 'tcl', 'Tools', and files like 'Untitled.ipynb', 'get-pip.py', 'LICENSE.txt', 'NEWS.txt', 'python.exe', 'pythonw.exe', 'README.txt', and 'w9xpopen.exe'. The right pane shows a list of files with columns for 'Name' and 'Last Modified'. The 'New' button is highlighted with a red box. A terminal window is open in the foreground, showing the command prompt 'C:\Python27>jupyter notebook' and the output 'Serving notebooks from local directory: C:\Python27'. The terminal also shows the Jupyter NotebookApp logs, including the URL 'http://localhost:8888/?token=5b172884d00d5a04ad53918771b14f48563c939e7ec53f26' and the message 'Accepting one-time-token-authenticated connection'.

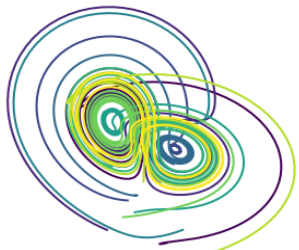
PAQUETES PARA TRADERS EN PYTHON

jupyter Atractor de Lorenz Last Checkpoint: hace 2 minutos (autosaved)  Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 2

```
def lorenz_deriv(x_y_z, t0, sigma=sigma, beta=beta, rho=rho):  
    """Compute the time-derivative of a Lorenz system."""  
    x, y, z = x_y_z  
    return [sigma * (y - x), x * (rho - z) - y, x * y - beta * z]  
  
# Choose random starting points, uniformly distributed from -15 to 15  
np.random.seed(1)  
x0 = -15 + 30 * np.random.random((N, 3))  
  
# Solve for the trajectories  
t = np.linspace(0, max time, int(250*max time))  
x_t = np.asarray([integrate.odeint(lorenz_deriv, x0i, t)  
                  for x0i in x0])  
  
# choose a different color for each trajectory  
colors = plt.cm.viridis(np.linspace(0, 1, N))  
  
for i in range(N):  
    x, y, z = x_t[i, :, :].T  
    lines = ax.plot(x, y, z, '-', c=colors[i])  
    plt.setp(lines, linewidth=2)  
  
ax.view_init(30, angle)  
plt.show()  
  
return t, x_t
```

In [2]: t, x_t = solve_lorenz(angle=0, N=10)





PRIMEROS PASOS EN PYTHON

PRIMEROS PASOS EN PYTHON

Instalamos Quandl:

```
pip install quandl
```

Ejecutamos una consulta:

```
import quandl
quandl.ApiConfig.api_key = 'WgQVD8Npfox1MN7SV3Nm'

aapl = quandl.get("WIKI/AAPL", start_date="1980-01-01", end_date="2017-12-31")
aapl
```

PRIMEROS PASOS EN PYTHON

9341 rows × 12 columns

```
In [6]: import quandl  
aapl = quandl.get("WIKI/AAPL", start_date="1980-01-01", end_date="2017-12-31")  
aapl
```

Out[6]:

	Open	High	Low	Close	Volume	Ex-Dividend	Split Ratio	Adj. Open	Adj. High	Adj. Low	Adj. Close	Adj. Volume
Date												
1980-12-12	28.75	28.8700	28.7500	28.750	2093900.0	0.0	1.0	0.422706	0.424470	0.422706	0.422706	117258400.0
1980-12-15	27.38	27.3800	27.2500	27.250	785200.0	0.0	1.0	0.402563	0.402563	0.400652	0.400652	43971200.0
1980-12-16	25.37	25.3700	25.2500	25.250	472000.0	0.0	1.0	0.373010	0.373010	0.371246	0.371246	26432000.0
1980-12-17	25.87	26.0000	25.8700	25.870	385900.0	0.0	1.0	0.380362	0.382273	0.380362	0.380362	21610400.0
1980-12-18	26.63	26.7500	26.6300	26.630	327900.0	0.0	1.0	0.391536	0.393300	0.391536	0.391536	18362400.0
1980-12-19	28.25	28.3800	28.2500	28.250	217100.0	0.0	1.0	0.415355	0.417266	0.415355	0.415355	12157600.0
1980-12-22	29.63	29.7500	29.6300	29.630	166800.0	0.0	1.0	0.435644	0.437409	0.435644	0.435644	9340800.0
1980-12-23	30.88	31.0000	30.8800	30.880	209600.0	0.0	1.0	0.454023	0.455787	0.454023	0.454023	11737600.0
1980-12-24	32.50	32.6300	32.5000	32.500	214300.0	0.0	1.0	0.477841	0.479753	0.477841	0.477841	12000800.0
1980-12-26	35.50	35.6200	35.5000	35.500	248100.0	0.0	1.0	0.521950	0.523714	0.521950	0.521950	13893600.0
1980-12-29	36.00	36.1300	36.0000	36.000	415900.0	0.0	1.0	0.529301	0.531213	0.529301	0.529301	23290400.0
1980-12-30	35.25	35.2500	35.1200	35.120	307500.0	0.0	1.0	0.518274	0.518274	0.516363	0.516363	17220000.0
1980-12-31	34.25	34.2500	34.1300	34.130	159600.0	0.0	1.0	0.503571	0.503571	0.501807	0.501807	8937600.0
1981-01-02	34.50	34.7500	34.5000	34.500	96700.0	0.0	1.0	0.507247	0.510923	0.507247	0.507247	5415200.0
1981-01-05	33.87	33.8700	33.7500	33.750	159500.0	0.0	1.0	0.497984	0.497984	0.496220	0.496220	8932000.0
1981-01-06	32.37	32.3700	32.2500	32.250	201600.0	0.0	1.0	0.475930	0.475930	0.474166	0.474166	11289600.0

PRIMEROS PASOS EN PYTHON

Seleccionamos fechas:

```
import pandas as pd
aapl2017 = aapl.loc[pd.Timestamp('2017-01-01'):pd.Timestamp('2017-12-31')]
aapl2017
```

Out[6]:

	Open	High	Low	Close	Volume	Ex-Dividend	Split Ratio	Adj. Open	Adj. High	Adj. Low	Adj. Close	Adj. Volume
Date												
2017-01-03	115.800	116.3300	114.7600	116.150	28781865.0	0.00	1.0	114.369701	114.893155	113.342546	114.715378	28781865.0
2017-01-04	115.850	116.5100	115.7500	116.020	21118116.0	0.00	1.0	114.419083	115.070931	114.320318	114.586983	21118116.0
2017-01-05	115.920	116.8642	115.8100	116.610	22193587.0	0.00	1.0	114.488219	115.420756	114.379577	115.169696	22193587.0
2017-01-06	116.780	118.1600	116.4700	117.910	31751900.0	0.00	1.0	115.337596	116.700551	115.031425	116.453639	31751900.0
2017-01-09	117.950	119.4300	117.9400	118.990	33561948.0	0.00	1.0	116.493145	117.954865	116.483269	117.520300	33561948.0
2017-01-10	118.770	119.3800	118.3000	119.110	24462051.0	0.00	1.0	117.303017	117.905483	116.838822	117.638817	24462051.0
2017-01-11	118.740	119.9300	118.6000	119.750	27588593.0	0.00	1.0	117.273388	118.448689	117.135117	118.270913	27588593.0
2017-01-12	118.895	119.3000	118.2100	119.250	27086220.0	0.00	1.0	117.426473	117.826471	116.749934	117.777088	27086220.0
2017-01-13	119.110	119.6200	118.8100	119.040	26111948.0	0.00	1.0	117.638817	118.142518	117.342523	117.569682	26111948.0
2017-01-17	118.340	120.2400	118.2200	120.000	34439843.0	0.00	1.0	116.878328	118.754860	116.759810	118.517825	34439843.0
2017-01-18	120.000	120.5000	119.7100	119.990	23712961.0	0.00	1.0	118.517825	119.011649	118.231407	118.507948	23712961.0
2017-01-19	119.400	120.0900	119.3700	119.780	25597291.0	0.00	1.0	117.925236	118.606713	117.895606	118.300542	25597291.0
2017-01-20	120.450	120.4500	119.7346	120.000	32597892.0	0.00	1.0	118.962267	118.962267	118.255703	118.517825	32597892.0
2017-01-23	120.000	120.8100	119.7700	120.080	22050218.0	0.00	1.0	118.517825	119.317820	118.290666	118.596837	22050218.0
2017-01-24	119.550	120.1000	119.5000	119.970	23211038.0	0.00	1.0	118.073383	118.616590	118.024000	118.488195	23211038.0
2017-01-25	120.420	122.1000	120.2800	121.880	32586673.0	0.00	1.0	118.932637	120.591887	118.794366	120.374604	32586673.0

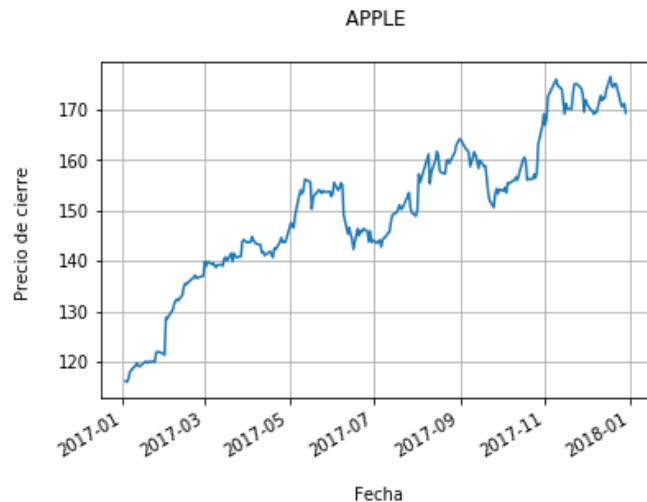
PRIMEROS PASOS EN PYTHON

Representamos gráficamente:

```
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize']=[50.0, 50.0]
aapl2017['Close'].plot(grid=True)
plt.title('' APPLE \n'')
plt.xlabel('\n Fecha')
plt.ylabel('Precio de cierre \n')
plt.show()
```

PRIMEROS PASOS EN PYTHON

```
In [8]: import matplotlib.pyplot as plt
aapl2017['Close'].plot(grid=True)
plt.title('' APPLE \n'')
plt.xlabel('\n Fecha')
plt.ylabel('Precio de cierre \n')
plt.show()
```



PRIMEROS PASOS EN PYTHON

Unas velitas, por favor:

```
from plotly import __version__
from plotly.offline import download_plotlyjs, init_notebook_mode, plot, iplot
import plotly.graph_objs as go
init_notebook_mode(connected=True)

trace = go.Candlestick(x=aapl2017.index, open=aapl2017.Open, high=aapl2017.High,
low=aapl2017.Low, close=aapl2017.Close)

data = [trace]

iplot(data, filename='simple_candlestick')
```

PRIMEROS PASOS EN PYTHON



PRIMEROS PASOS EN PYTHON

Generamos rendimientos porcentuales y logarítmicos:

```
import numpy as np
daily_close = aapl[['Close']]
daily_pct_change = daily_close.pct_change()

# Reemplazamos NA por 0
daily_pct_change.fillna(0, inplace=True)

print(daily_pct_change)

daily_log_returns = np.log(daily_close.pct_change()+1)

print(daily_log_returns)
```

PRIMEROS PASOS EN PYTHON

Distribución de rendimientos y principales datos estadísticos de la distribución:

```
import matplotlib.pyplot as plt

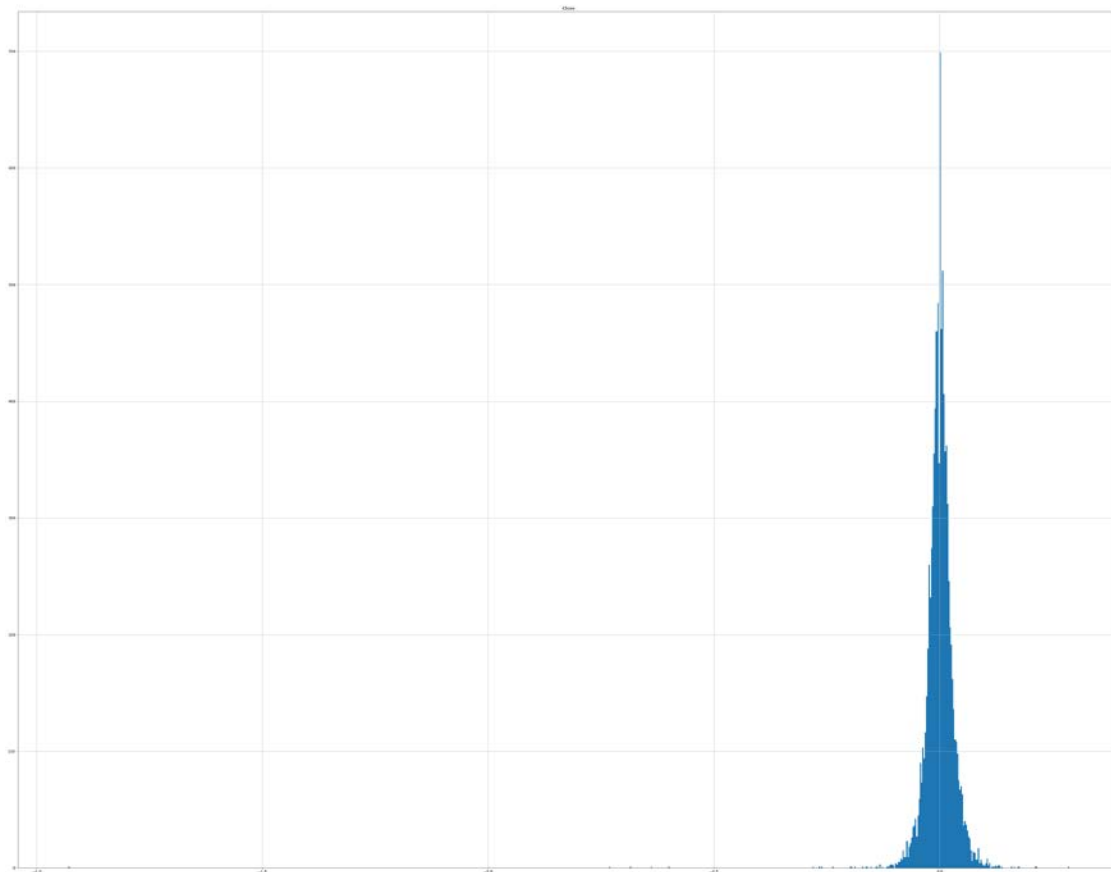
plt.rcParams['figure.figsize']=[50.0, 50.0]

daily_log_returns.hist(bins='auto')

plt.show()

print(daily_log_returns.describe())
```

PRIMEROS PASOS EN PYTHON



	Close
count	9340.000000
mean	0.000190
std	0.037471
min	-1.930035
25%	-0.013866
50%	0.000000
75%	0.014927
max	0.286796

PRIMEROS PASOS EN PYTHON

Múltiples activos:

```
import quandl
import pandas as pd
quandl.ApiConfig.api_key = 'WgQVD8Npfox1MN7SV3Nm'

axp = quandl.get("WIKI/AXP", start_date="2000-01-01", end_date="2017-12-31",
returns="pandas")
bac = quandl.get("WIKI/BAC", start_date="2000-01-01", end_date="2017-12-31",
returns="pandas")
cit = quandl.get("WIKI/C", start_date="2000-01-01", end_date="2017-12-31",
returns="pandas")
gs = quandl.get("WIKI/GS", start_date="2000-01-01", end_date="2017-12-31",
returns="pandas")
jpm = quandl.get("WIKI/JPM", start_date="2000-01-01", end_date="2017-12-31",
returns="pandas")

portfolio = pd.concat([axp.Close, bac.Close, cit.Close, gs.Close, jpm.Close],
axis=1, join_axes=[axp.Close.index])
portfolio.columns = ['AXP', 'BAC', 'C', 'GS', 'JPM']
portfolio
```

PRIMEROS PASOS EN PYTHON

```
In [1]: import quandl
import pandas as pd
axp = quandl.get("WIKI/AXP", start_date="2000-01-01", end_date="2017-12-31", returns="pandas")
bac = quandl.get("WIKI/BAC", start_date="2000-01-01", end_date="2017-12-31", returns="pandas")
cit = quandl.get("WIKI/C", start_date="2000-01-01", end_date="2017-12-31", returns="pandas")
gs = quandl.get("WIKI/GS", start_date="2000-01-01", end_date="2017-12-31", returns="pandas")
jpm = quandl.get("WIKI/JPM", start_date="2000-01-01", end_date="2017-12-31", returns="pandas")

portfolio = pd.concat([axp.Close, bac.Close, cit.Close, gs.Close, jpm.Close], axis=1, join_axes=[axp.Close.index])
portfolio.columns = ['AXP', 'BAC', 'C', 'GS', 'JPM']
portfolio
```

Out[1]:

	AXP	BAC	C	GS	JPM
Date					
2000-01-03	157.25	48.44	53.00	88.31	72.87
2000-01-04	151.30	45.56	49.75	82.75	70.88
2000-01-05	150.30	46.06	51.75	79.69	70.44
2000-01-06	150.30	50.00	54.25	82.25	71.44
2000-01-07	152.40	48.69	54.00	82.56	72.75
2000-01-10	154.60	47.00	53.81	84.38	71.50
2000-01-11	156.00	46.00	53.13	84.81	69.81
2000-01-12	155.20	46.50	55.00	84.38	70.25
2000-01-13	156.00	47.56	55.63	84.50	71.31
2000-01-14	159.50	50.50	58.00	85.63	73.88
2000-01-18	151.60	48.00	58.50	86.88	71.00
2000-01-19	148.90	48.00	59.75	87.56	74.13
2000-01-20	151.40	47.08	58.61	85.94	74.06
2000-01-21	151.70	45.69	56.44	85.69	72.94

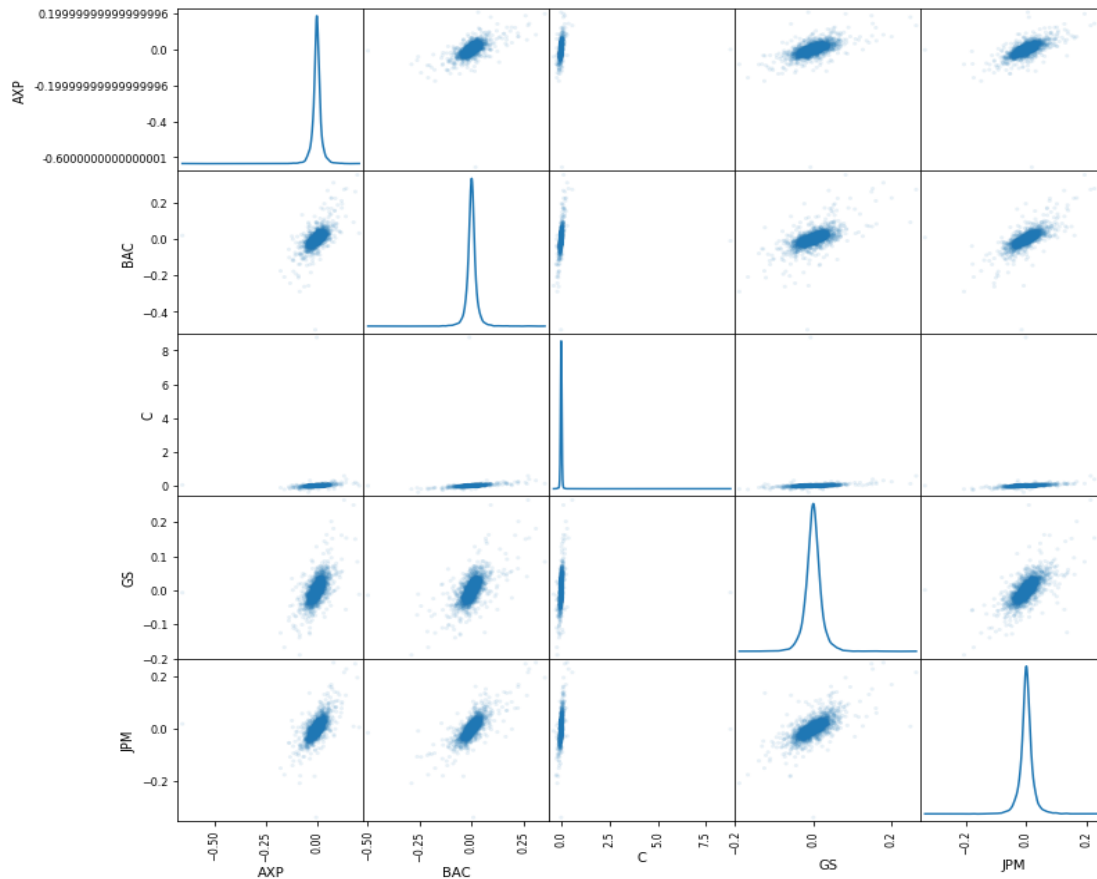
PRIMEROS PASOS EN PYTHON

Correlaciones entre activos:

```
portfolio_returns = portfolio.pct_change()  
portfolio_returns
```

```
import matplotlib.pyplot as plt  
plt.rcParams['figure.figsize']=[50.0, 50.0]  
pd.plotting.scatter_matrix(portfolio_returns,diagonal='kde', alpha=0.1,  
figsize=(12,12))  
plt.show()
```

PRIMEROS PASOS EN PYTHON



PRIMEROS PASOS EN PYTHON

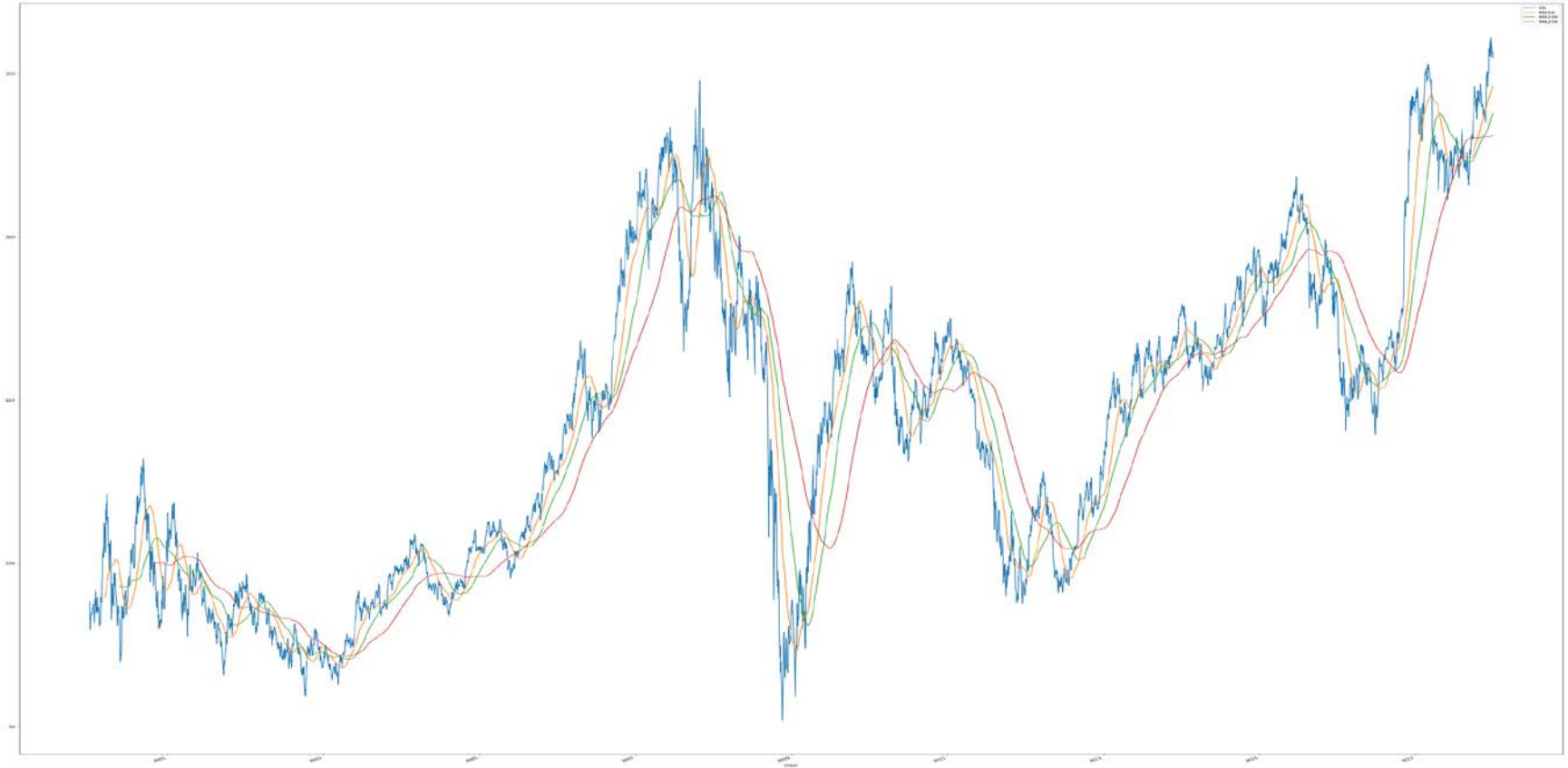
Medias Móviles:

```
goldman = portfolio.GS
ma50 = goldman.rolling(window=50).mean()
ma100 = goldman.rolling(window=100).mean()
ma200 = goldman.rolling(window=200).mean()

df = pd.concat([goldman,ma50,ma100,ma200], axis=1, join_axes=[goldman.index])
df.columns = ['GS', 'MA50', 'MA100', 'MA200']
df

import matplotlib.pyplot as plt
plt.rcParams['figure.figsize']=[50.0, 50.0]
plt.figure()
df.plot()
plt.show()
```

PRIMEROS PASOS EN PYTHON



PRIMEROS PASOS EN PYTHON

Regresion Lineal:

```
import numpy as np
import statsmodels.api as sm

jpmorgan = np.array(portfolio_returns.JPM)
bac = np.array(portfolio_returns.BAC)

#Quitamos NaN
x_clean = [x for x in jpmorgan if str(x) != 'nan']
y_clean = [x for x in bac if str(x) != 'nan']

#Añadimos constante
x_cte = sm.add_constant(x_clean, prepend=False)

model = sm.OLS(y_clean, x_cte)
res = model.fit()
print(res.summary())
```

PRIMEROS PASOS EN PYTHON

OLS Regression Results

```
=====
Dep. Variable:          y      R-squared:          0.539
Model:                  OLS    Adj. R-squared:      0.538
Method:                 Least Squares    F-statistic:      5279.
Date:                   Mon, 05 Feb 2018    Prob (F-statistic):    0.00
Time:                   19:40:01    Log-Likelihood:      11090.
No. Observations:      4526    AIC:                  -2.218e+04
Df Residuals:          4524    BIC:                  -2.216e+04
Df Model:               1
Covariance Type:       nonrobust
=====
```

```
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
x1              0.8778      0.012      72.656      0.000      0.854      0.901
const          1.029e-05      0.000       0.033      0.974     -0.001      0.001
=====
```

```
=====
Omnibus:                 3121.202    Durbin-Watson:           1.897
Prob(Omnibus):            0.000    Jarque-Bera (JB):        1586847.928
Skew:                     -2.028    Prob(JB):                0.00
Kurtosis:                 94.641    Cond. No.                38.9
=====
```

Warnings:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

PRIMEROS PASOS EN PYTHON

Representamos nube de puntos y recta:

```
import seaborn as sns
```

```
fig.set_size_inches(11.7, 8.27)
```

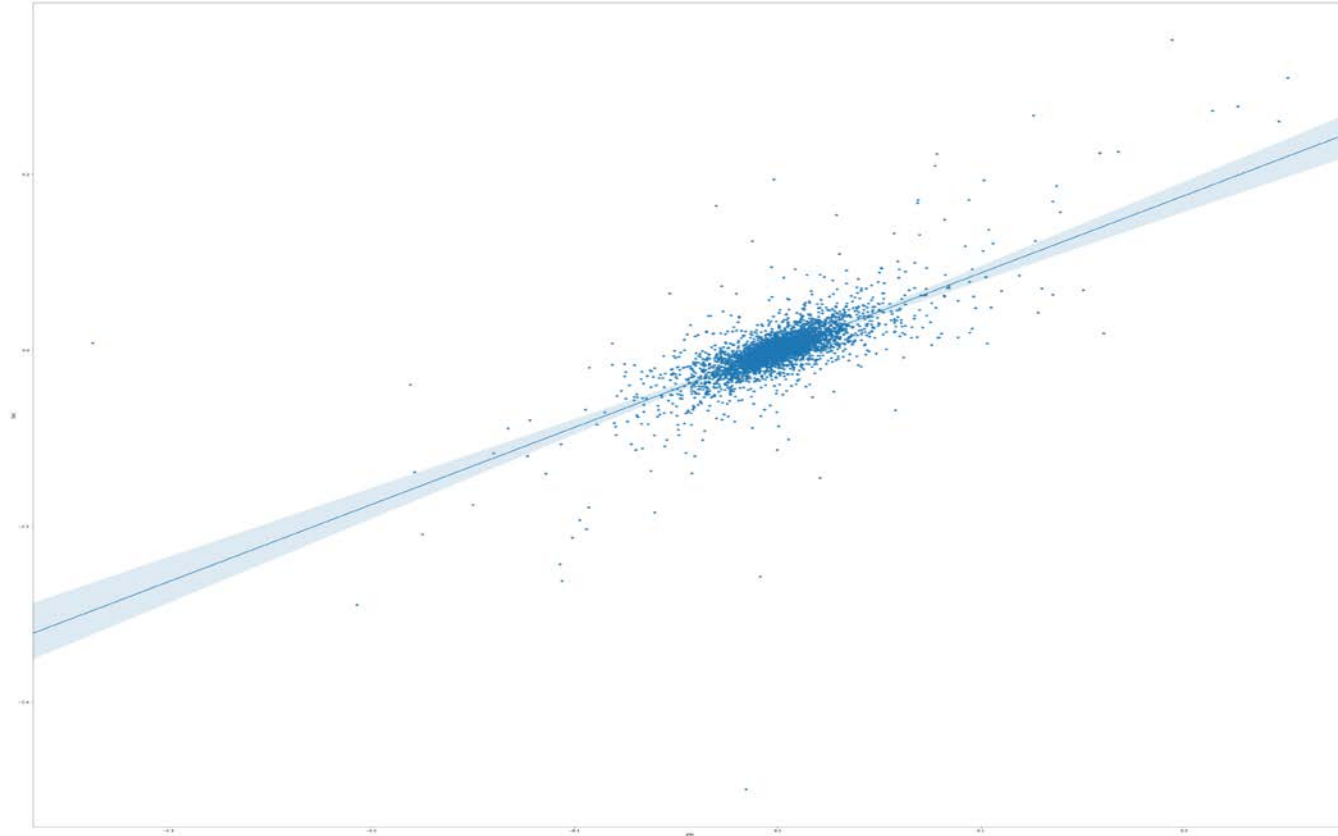
```
eje_x = portfolio_returns.JPM
```

```
eje_y = portfolio_returns.BAC
```

```
df_ret = pd.concat([eje_x, eje_y],axis=1, join_axes=[eje_x.index])  
df_ret
```

```
sns.regplot(x='JPM', y='BAC', data=df_ret)
```

PRIMEROS PASOS EN PYTHON



PRIMEROS PASOS EN PYTHON

Correlaciones:

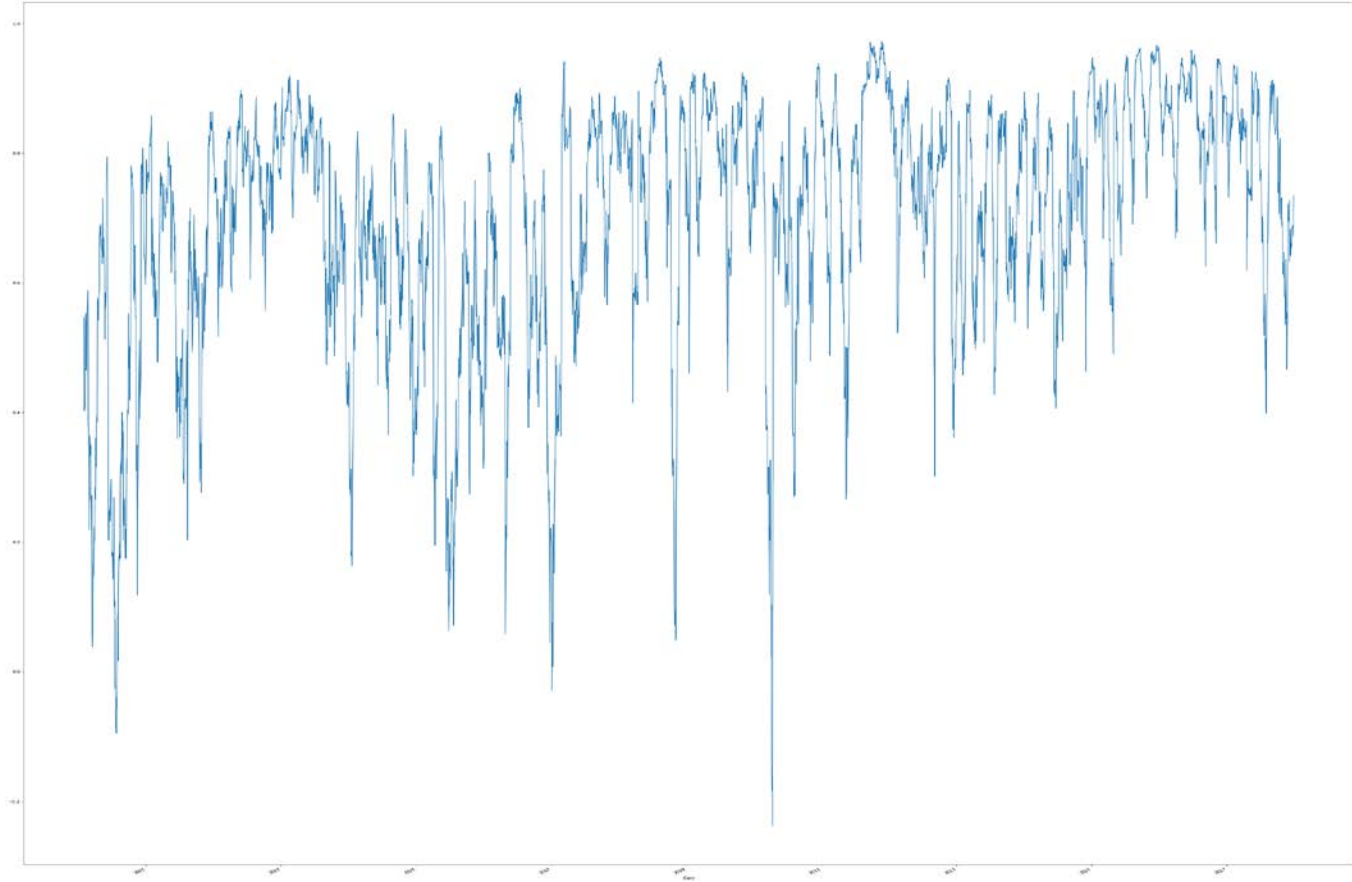
```
import pandas as pd
import matplotlib.pyplot as plt

corr_bancos =
portfolio_returns.JPM.rolling(window=20).corr(other=portfolio_returns.GS)

corr_bancos

plt.rcParams['figure.figsize']=[50.0, 50.0]
corr_bancos.plot()
plt.show()
```

PRIMEROS PASOS EN PYTHON



PRIMEROS PASOS EN PYTHON

Simulaciones de Monte Carlo

```
import numpy as np
import math
import matplotlib.pyplot as plt
from scipy.stats import norm
import quandl
quandl.ApiConfig.api_key = 'WgQVD8Npfox1MN7SV3Nm'
```

```
msft = quandl.get("WIKI/MSFT", start_date="1980-01-01", end_date="2017-12-31")
```

```
msft['Returns'] = msft['Close'].pct_change()
mean = msft['Returns'].mean()
vol = msft['Returns'].std()*math.sqrt(252)
```

PRIMEROS PASOS EN PYTHON

Simulaciones de Monte Carlo

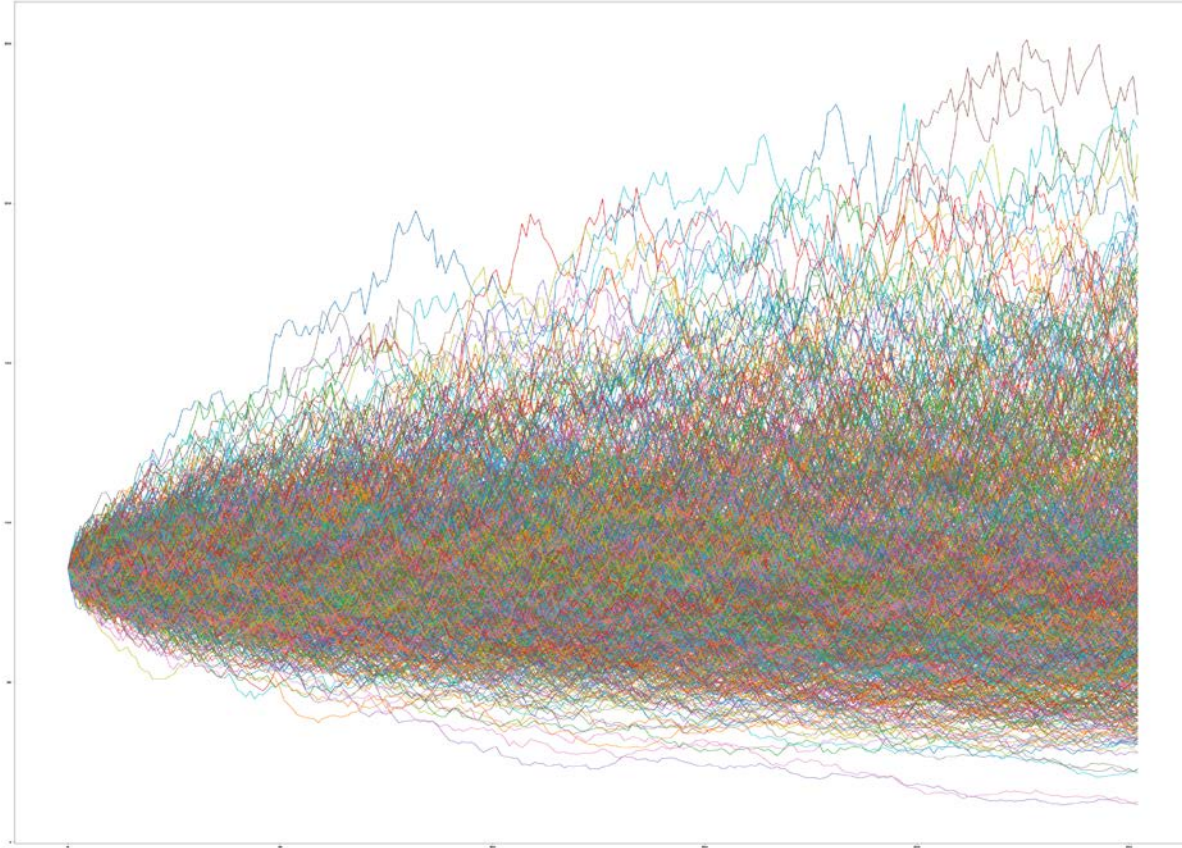
```
S = msft['Close'][-1]
T = 252

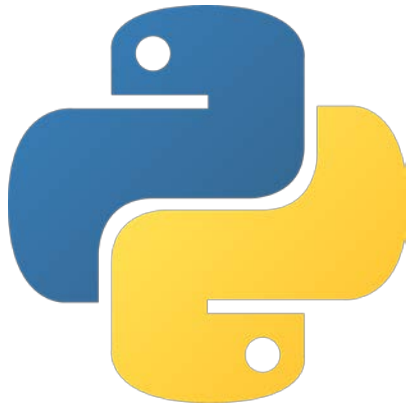
for i in range(1000):
    daily_returns=np.random.normal(mean/T,vol/math.sqrt(T),T)+1
    price_list = [S]

    for x in daily_returns:
        price_list.append(price_list[-1]*x)

plt.rcParams['figure.figsize']=[50.0, 50.0]
plt.plot(price_list)
plt.show()
```

PRIMEROS PASOS EN PYTHON





MI PRIMER SISTEMA DE TRADING EN PYTHON

MI PRIMER SISTEMA DE TRADING EN PYTHON

Sistema Cruce Medias Móviles

Reglas del Sistema

- Compramos cada vez que la SMA(18) cruza al alza a la SMA(40).
- Vendemos cada vez que la SMA(18) cruza a la baja a la SMA(40).

MI PRIMER SISTEMA DE TRADING EN PYTHON

Sistema Cruce Medias Móviles

```
#Cargamos librerías y descargamos datos
import quandl
import pandas as pd
import numpy as np

quandl.ApiConfig.api_key = 'WgQVD8Npfox1MN7SV3Nm'

ebay = quandl.get("WIKI/EBAY", start_date="2000-01-01", end_date="2017-12-31",
returns="pandas")

#Definimos períodos
short_window = 18
long_window = 40

# Inicializamos señales con un Dataframe
signals = pd.DataFrame(index=ebay.index)
signals['signal'] = 0.0
```

MI PRIMER SISTEMA DE TRADING EN PYTHON

Sistema Cruce Medias Móviles

```
#Creamos medias móviles y generamos señales de cruce
```

```
signals['short_mavg'] = ebay['Close'].rolling(window=short_window, min_periods=1,  
center=False).mean()
```

```
signals['long_mavg'] = ebay['Close'].rolling(window=long_window, min_periods=1,  
center=False).mean()
```

```
signals['signal'][short_window:] = np.where(signals['short_mavg'][short_window:]  
> signals['long_mavg'][short_window:], 1.0, 0.0)
```

```
signals['positions'] = signals['signal'].diff()
```

```
#Mostramos señales  
print(signals)
```

MI PRIMER SISTEMA DE TRADING EN PYTHON

Date	signal	short_mavg	long_mavg	positions
2000-02-14	0.0	147.763333	147.763333	0.0
...	...	...	...	...
2017-11-16	1.0	37.512000	36.641500	0.0
2017-11-17	1.0	37.445250	36.648200	0.0
2017-11-20	1.0	37.376250	36.650400	0.0
2017-11-21	1.0	37.330750	36.664900	0.0
2017-11-22	1.0	37.283000	36.675100	0.0
2017-11-24	1.0	37.227750	36.687500	0.0
2017-11-27	1.0	37.184250	36.700200	0.0
2017-11-28	1.0	37.110250	36.714900	0.0
2017-11-29	1.0	37.001500	36.716400	0.0
2017-11-30	1.0	36.900500	36.712700	0.0
2017-12-01	1.0	36.814000	36.712800	0.0
2017-12-04	1.0	36.731750	36.709500	0.0
2017-12-05	0.0	36.681750	36.712600	-1.0
2017-12-06	0.0	36.627750	36.710100	0.0
2017-12-07	0.0	36.578000	36.708100	0.0
2017-12-08	0.0	36.561500	36.716600	0.0
2017-12-11	0.0	36.547250	36.721000	0.0
2017-12-12	0.0	36.524500	36.723600	0.0
2017-12-13	0.0	36.515250	36.731500	0.0
2017-12-14	0.0	36.518000	36.743600	0.0
2017-12-15	0.0	36.528000	36.764100	0.0
2017-12-18	0.0	36.550250	36.775500	0.0
2017-12-19	0.0	36.555500	36.792000	0.0
2017-12-20	0.0	36.583000	36.811900	0.0
2017-12-21	0.0	36.609750	36.832700	0.0
2017-12-22	0.0	36.636750	36.851200	0.0
2017-12-26	0.0	36.662750	36.871600	0.0
2017-12-27	0.0	36.683000	36.890600	0.0
2017-12-28	0.0	36.706000	36.913600	0.0
2017-12-29	0.0	36.708500	36.929900	0.0

[4527 rows x 4 columns]

MI PRIMER SISTEMA DE TRADING EN PYTHON

Gráfico de las Operaciones

```
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize']=[50.0, 50.0]
fig = plt.figure()

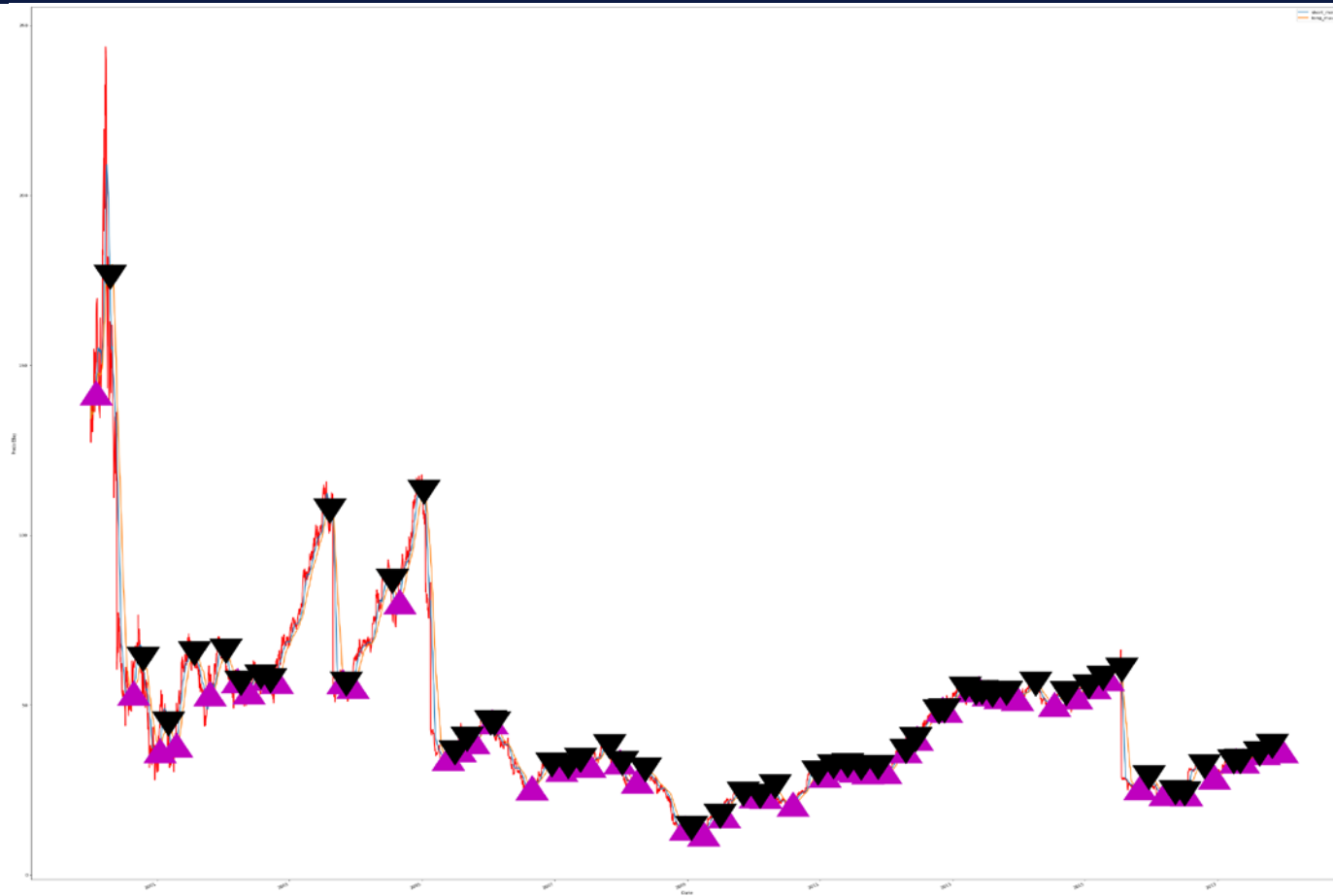
ax1 = fig.add_subplot(111,  ylabel='Precio EBay')
ebay['Close'].plot(ax=ax1, color='r', lw=2.)
signals[['short_mavg', 'long_mavg']].plot(ax=ax1, lw=2.)

# Señales de Compra
ax1.plot(signals.loc[signals.positions == 1.0].index,
        signals.short_mavg[signals.positions == 1.0],
        '^', markersize=45, color='m')

# Señales de Venta
ax1.plot(signals.loc[signals.positions == -1.0].index,
        signals.short_mavg[signals.positions == -1.0],
        'v', markersize=45, color='k')

plt.show()
```

MI PRIMER SISTEMA DE TRADING EN PYTHON



MI PRIMER SISTEMA DE TRADING EN PYTHON

Un Sencillo Backtest

```
#Definimos capital y posiciones
initial_capital= float(100000.0)
positions = pd.DataFrame(index=signals.index).fillna(0.0)

#Fijamos tamaño de la posición a 100 acciones
positions['EBAY'] = 100*signals['signal']

#Inicializamos portfolio
portfolio = positions.multiply(ebay['Close'], axis=0)

#Almacenamos diferencias en las posiciones
pos_diff = positions.diff()
```

MI PRIMER SISTEMA DE TRADING EN PYTHON

Un Sencillo Backtest

```
#Calculamos posiciones mantenidas
portfolio['holdings'] = (positions.multiply(ebay['Close'], axis=0)).sum(axis=1)

#Obtenemos capital que nos queda tras abrir posición
portfolio['cash'] = initial_capital - (pos_diff.multiply(ebay['Close'],
axis=0)).sum(axis=1).cumsum()

#Sumamos cash + acciones
portfolio['total'] = portfolio['cash'] + portfolio['holdings']

#Calculamos rentabilidades y vemos resultados
portfolio['returns'] = portfolio['total'].pct_change()

portfolio
```


MI PRIMER SISTEMA DE TRADING EN PYTHON

Out[5]:

Date	EBAY	holdings	cash	total	returns
2017-12-01	0.0	0.0	107967.0	107967.0	0.000000
2017-12-04	0.0	0.0	107967.0	107967.0	0.000000
2017-12-05	0.0	0.0	107967.0	107967.0	0.000000
2017-12-06	0.0	0.0	107967.0	107967.0	0.000000
2017-12-07	0.0	0.0	107967.0	107967.0	0.000000
2017-12-08	0.0	0.0	107967.0	107967.0	0.000000
2017-12-11	0.0	0.0	107967.0	107967.0	0.000000
2017-12-12	0.0	0.0	107967.0	107967.0	0.000000
2017-12-13	0.0	0.0	107967.0	107967.0	0.000000
2017-12-14	0.0	0.0	107967.0	107967.0	0.000000
2017-12-15	0.0	0.0	107967.0	107967.0	0.000000
2017-12-18	0.0	0.0	107967.0	107967.0	0.000000
2017-12-19	3782.0	3782.0	104185.0	107967.0	0.000000
2017-12-20	3793.0	3793.0	104185.0	107978.0	0.000102
2017-12-21	3781.0	3781.0	104185.0	107966.0	-0.000111
2017-12-22	3776.0	3776.0	104185.0	107961.0	-0.000046
2017-12-26	3794.0	3794.0	104185.0	107979.0	0.000167
2017-12-27	3761.0	3761.0	104185.0	107946.0	-0.000306
2017-12-28	3792.0	3792.0	104185.0	107977.0	0.000287
2017-12-29	3774.0	3774.0	104185.0	107959.0	-0.000167

4527 rows × 5 columns

MI PRIMER SISTEMA DE TRADING EN PYTHON

Gráfico de la Equity

```
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize']=[50.0, 50.0]

fig = plt.figure()

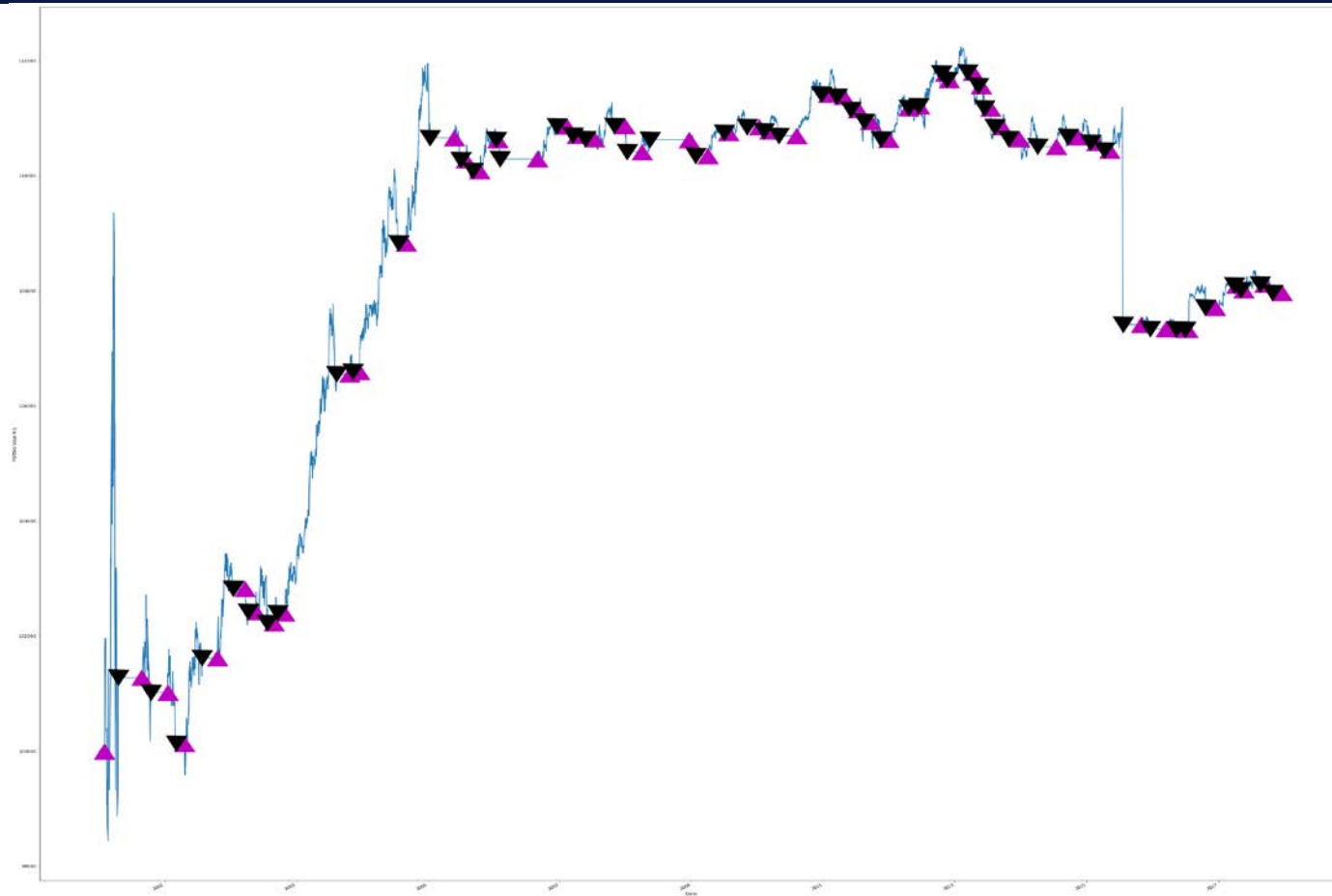
ax1 = fig.add_subplot(111, ylabel='Portfolio value in $')

# Dibujamos la equity en dólares
portfolio['total'].plot(ax=ax1, lw=2.)

ax1.plot(portfolio.loc[signals.positions == 1.0].index,
         portfolio.total[signals.positions == 1.0],
         '^', markersize=45, color='m')
ax1.plot(portfolio.loc[signals.positions == -1.0].index,
         portfolio.total[signals.positions == -1.0],
         'v', markersize=45, color='k')

plt.show()
```

MI PRIMER SISTEMA DE TRADING EN PYTHON



MI PRIMER SISTEMA DE TRADING EN PYTHON

Evaluando la Estrategia

#Ratio de Sharpe

```
returns = portfolio['returns']
```

```
sharpe_ratio = np.sqrt(252) * (returns.mean() / returns.std())
```

```
print(sharpe_ratio)
```

```
In [7]: #Ratio de Sharpe

returns = portfolio['returns']

sharpe_ratio = np.sqrt(252) * (returns.mean() / returns.std())

print(sharpe_ratio)
```

0.16881507559808137

MI PRIMER SISTEMA DE TRADING EN PYTHON

Evaluando la Estrategia

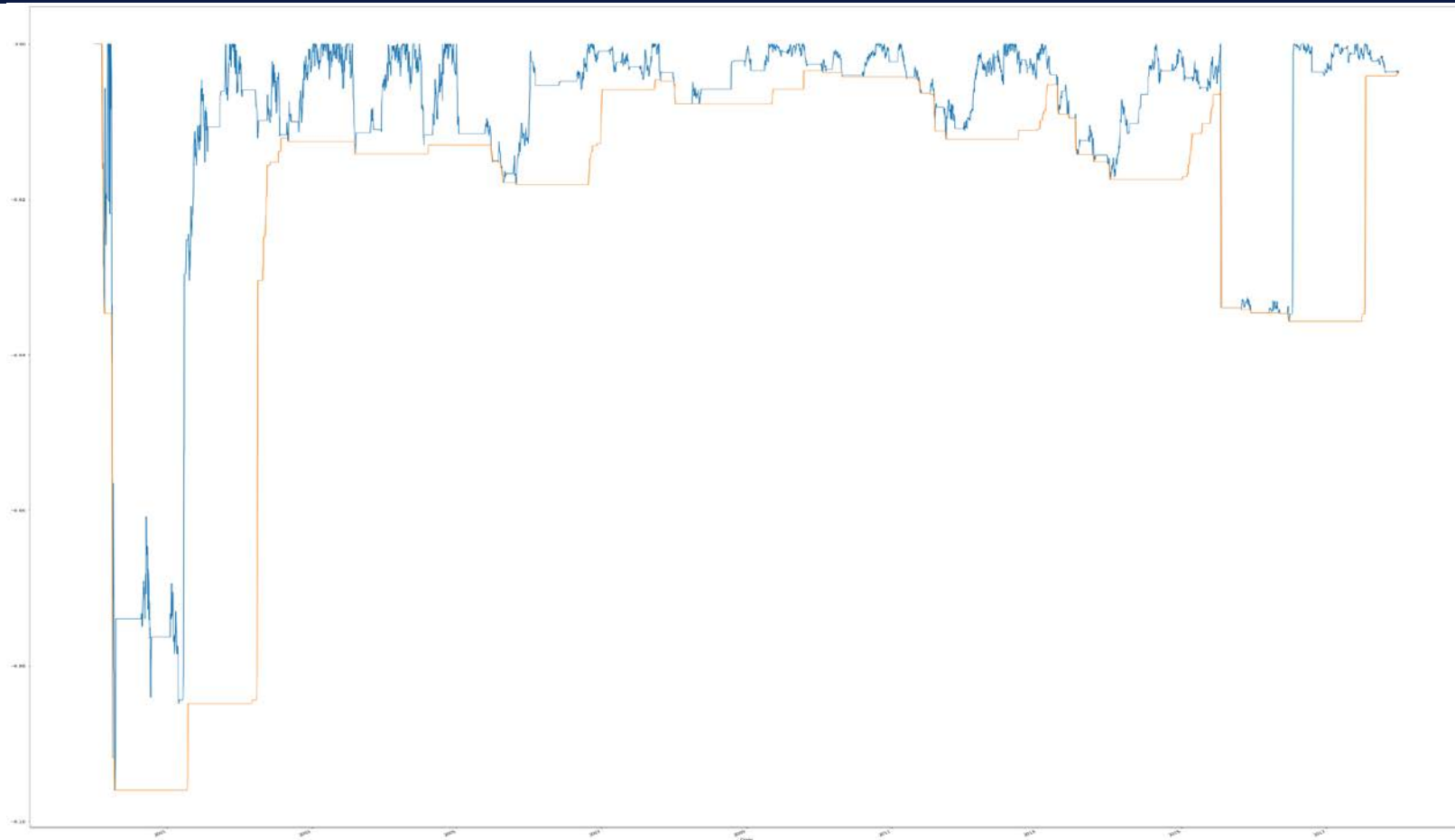
#Análisis del Drawdown

```
window = 252
rolling_max = portfolio['total'].rolling(window, min_periods=1).max()
daily_drawdown = portfolio['total']/rolling_max - 1.0
max_daily_drawdown = daily_drawdown.rolling(window, min_periods=1).min()

daily_drawdown.plot()
max_daily_drawdown.plot()

plt.show()
```

MI PRIMER SISTEMA DE TRADING EN PYTHON



MI PRIMER SISTEMA DE TRADING EN PYTHON

Optimizando la Estrategia

```
import quandl
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

quandl.ApiConfig.api_key = 'WgQVD8Npfox1MN7SV3Nm'

def ma_strat(ticker, short_ma, long_ma):
    datos = quandl.get(ticker, start_date="2000-01-01", end_date="2017-12-31")
    datos['short_ma'] = datos['Close'].rolling(window=short_ma).mean()
    datos['long_ma'] = datos['Close'].rolling(window=long_ma).mean()
    datos.dropna(inplace=True)
    datos['position'] = np.where(datos['short_ma'] > datos['long_ma'], 1, 0)
    datos['rets'] = np.log(datos['Close'] / datos['Close'].shift(1))
    datos['strategy'] = datos['rets'] * datos['position'].shift(1)
    sharpe = np.sqrt(252) * (datos['strategy'].mean() / datos['strategy'].std())
    rets = datos['strategy'].cumsum()[-1]
    return rets, sharpe
```

MI PRIMER SISTEMA DE TRADING EN PYTHON

Optimizando la Estrategia

```
#Definimos rango de períodos para las medias
shortp = np.linspace(10,30,20, dtype=int)
longp = np.linspace(40,60,20, dtype=int)

profit = np.zeros((len(shortp), len(longp)))
sharperatio = np.zeros((len(shortp), len(longp)))

#Iniciamos bucle 'for' para optimizar
ticker="WIKI/EBAY"
for i, s in enumerate(shortp):
    for j, l in enumerate(longp):
        rets, sharpe = ma_strat(ticker, s, l)
        profit[i,j] = rets
        sharperatio[i,j] = sharpe
```


MI PRIMER SISTEMA DE TRADING EN PYTHON

Optimizando la Estrategia

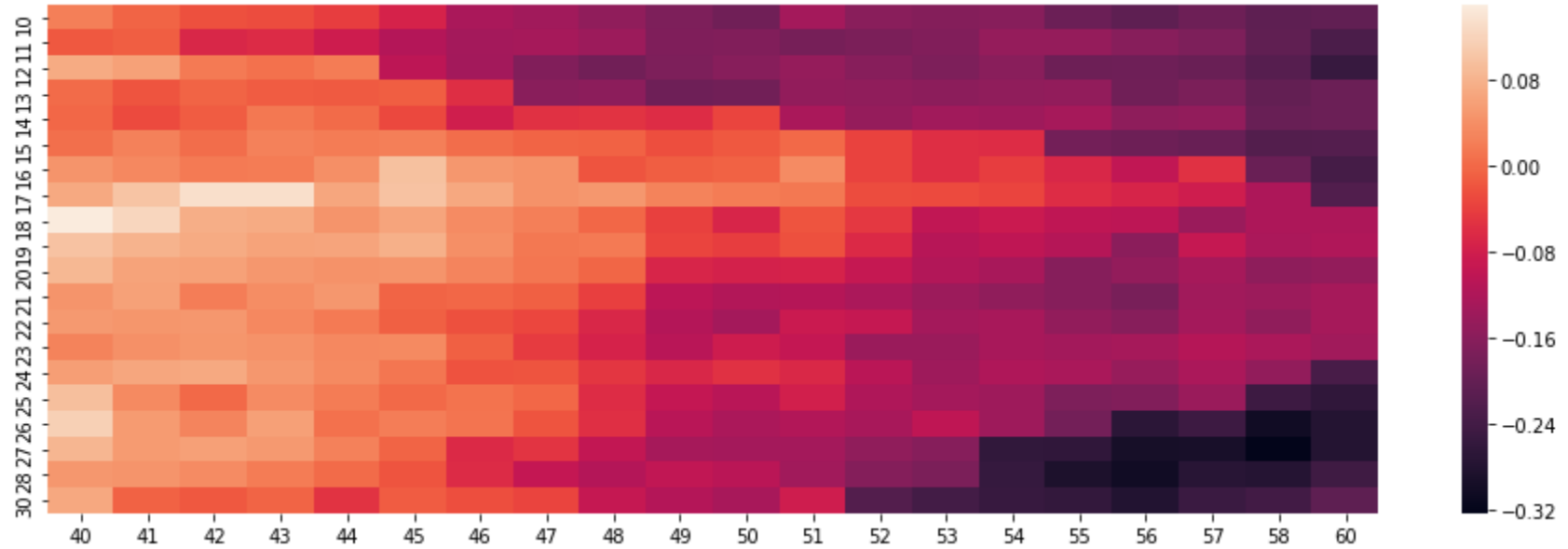
```
plt.figure(figsize = (16,5))

#Plot vs returns
sns.heatmap(profit, xticklabels=longp, yticklabels=shortp, annot=True,
linewidths=.5)

#Plot vs Sharpe
sns.heatmap(sharperatio, xticklabels=longp, yticklabels=shortp)
```

	-0.15	-0.031	-0.15	-0.19	-0.29	-0.49	-0.85	-0.9	-1	-1.2	-1.3	-0.88	-1.1	-1.1	-1.1	-1.3	-1.4	-1.3	-1.4	-1.3
	-0.1	-0.064	-0.45	-0.42	-0.57	-0.77	-0.88	-0.86	-0.94	-1.2	-1.1	-1.2	-1.2	-1.1	-0.99	-0.99	-1.1	-1.1	-1.4	-1.5
10	0.43	0.36	0.1	0.056	0.12	-0.7	-0.9	-1.1	-1.3	-1.2	-1.1	-0.99	-1.1	-1.2	-1.1	-1.3	-1.3	-1.4	-1.7	
11	0.013	-0.12	-0.031	-0.076	-0.087	-0.066	-0.36	-1.1	-1.1	-1.3	-1.3	-1	-1	-1.1	-1	-1	-1.2	-1.2	-1.3	-1.3
12	-0.0067	-0.18	-0.071	0.099	0.019	-0.18	-0.48	-0.34	-0.32	-0.36	-0.21	-0.85	-0.99	-0.9	-0.92	-0.85	-1	-1	-1.3	-1.3
13	0.045	0.16	0.033	0.16	0.12	0.14	0.032	-0.029	-0.04	-0.16	-0.1	0.002	-0.22	-0.35	-0.36	-1.2	-1.3	-1.3	-1.5	-1.5
14	0.27	0.19	0.1	0.12	0.24	0.6	0.3	0.27	-0.12	-0.063	-0.036	0.22	-0.22	-0.36	-0.25	-0.41	-0.6	-0.33	-1.3	-1.6
15	0.42	0.62	0.83	0.83	0.41	0.61	0.41	0.27	0.3	0.16	0.12	0.094	-0.16	-0.17	-0.2	-0.37	-0.43	-0.5	-0.74	-1.5
16	0.92	0.76	0.46	0.44	0.28	0.39	0.22	0.14	0.0066	-0.24	-0.43	-0.11	-0.29	-0.6	-0.53	-0.62	-0.63	-0.87	-0.74	-0.74
17	0.61	0.49	0.44	0.38	0.39	0.48	0.24	0.1	-0.21	-0.25	-0.14	-0.4	-0.67	-0.61	-0.68	-0.97	-0.58	-0.76	-0.71	
18	0.54	0.38	0.37	0.31	0.27	0.28	0.18	0.088	-0.012	-0.43	-0.46	-0.45	-0.57	-0.71	-0.78	-1	-0.91	-0.79	-0.95	-0.91
19	0.27	0.38	0.13	0.24	0.3	-0.027	-0.0026	-0.055	-0.24	-0.65	-0.71	-0.69	-0.76	-0.85	-0.94	-1	-1.1	-0.82	-0.85	-0.78
20	0.31	0.29	0.3	0.2	0.11	-0.047	-0.14	-0.2	-0.42	-0.7	-0.81	-0.54	-0.57	-0.81	-0.79	-0.93	-1	-0.79	-0.93	-0.79
21	0.17	0.26	0.29	0.26	0.2	0.21	-0.05	-0.27	-0.45	-0.66	-0.52	-0.59	-0.87	-0.88	-0.78	-0.82	-0.78	-0.69	-0.76	-0.82
22	0.36	0.41	0.43	0.31	0.21	0.085	-0.13	-0.12	-0.3	-0.42	-0.33	-0.41	-0.66	-0.86	-0.74	-0.77	-0.89	-0.76	-0.92	-1.6
23	0.59	0.21	0.0068	0.22	0.12	0.00075	0.063	0.0077	-0.37	-0.58	-0.68	-0.49	-0.74	-0.81	-0.85	-1.1	-1	-0.87	-1.7	-1.8
24	0.72	0.33	0.18	0.36	0.046	0.14	0.074	-0.12	-0.35	-0.68	-0.77	-0.72	-0.79	-0.62	-0.85	-1.1	-1.8	-1.7	-2	-1.9
25	0.52	0.33	0.36	0.32	0.16	-0.024	-0.4	-0.31	-0.61	-0.8	-0.81	-0.81	-0.95	-1	-1.8	-1.8	-2	-2	-2.2	-1.9
26	0.3	0.29	0.22	0.12	0.016	-0.12	-0.38	-0.6	-0.71	-0.61	-0.66	-0.83	-1	-1.1	-1.7	-2	-2	-1.9	-1.9	-1.7
27	0.42	-0.045	-0.1	-0.028	-0.33	-0.077	-0.16	-0.22	-0.58	-0.71	-0.8	-0.51	-1.5	-1.6	-1.7	-1.8	-1.9	-1.7	-1.6	-1.4
28	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	60

MI PRIMER SISTEMA DE TRADING EN PYTHON



MI PRIMER SISTEMA DE TRADING EN PYTHON

Veamos algo más avanzado: Quantiacs

- Quantiacs proporciona una plataforma para analizar y ejecutar algoritmos de trading. Posteriormente la compañía se encarga de poner en contacto a hedge funds y quants.
- Dispone de un excelente toolkit para Python que podemos instalar tecleando:

```
pip install quantiacsToolbox
```

- Permite almacenar cada estrategia en un archivo .py individual.
- Una vez desarrollada la estrategia podemos enviarla a Quantiacs y participar en competiciones de 3 meses. Los 3 mejores algoritmos reciben una inversión de \$1.000.000, \$750.000 y \$500.000

MI PRIMER SISTEMA DE TRADING EN PYTHON

Estrategia Mean Reversion con Quantiacs

Creamos la estrategia:

- Definimos el número de velas (*candles*) que consideraremos para generar la señal de compra o venta.
- Seguidamente calculamos la diferencia de precios de cierre de las últimas n candles. Si todas las diferencias de cierres son positivas abriremos una posición corta (-1), mientras que si todas las diferencias son negativas abriremos posición larga (+1)

MI PRIMER SISTEMA DE TRADING EN PYTHON

Estrategia Mean Reversion con Quantiacs

Definimos parámetros de la estrategia:

```
import numpy
```

```
def mySettings():  
    settings={}  
    settings['markets'] = ['BA', 'GS', 'MMM', 'UNH', 'HD']  
    settings['beginInSample'] = '20000101'  
    settings['endInSample'] = '20171231'  
    settings['lookback'] = 5  
    settings['budget'] = 10**6  
    settings['slippage'] = 0.05  
    return settings
```

MI PRIMER SISTEMA DE TRADING EN PYTHON

Estrategia Mean Reversion con Quantiacs

Definimos reglas de la estrategia:

```
def myTradingSystem(DATE, OPEN, HIGH, LOW, CLOSE, VOL, exposure, equity,
settings):
    nMarkets = CLOSE.shape[1]
    candles = 4
    lastncandles = CLOSE[-candles:,:]
    print lastncandles
    candlesDiff = numpy.diff(lastncandles, axis=0)
    print candlesDiff
    longEquity = numpy.all(candlesDiff<0, axis=0)
    print longEquity
    shortEquity = numpy.all(candlesDiff>0, axis=0)
    print shortEquity
    pos=numpy.zeros(nMarkets)
    pos[longEquity] = 1
    pos[shortEquity] = -1
    print pos
    return pos, settings
```

MI PRIMER SISTEMA DE TRADING EN PYTHON

Estrategia Mean Reversion con Quantiacs

Finalizamos estrategia:

```
if __name__ == '__main__':  
    import quantiacsToolbox  
    results = quantiacsToolbox.runts(__file__)
```

Ejecutamos estrategia:

```
import quantiacsToolbox  
results = quantiacsToolbox.runts('C:\\python27\\quantiacs\\candles.py')
```


MI PRIMER SISTEMA DE TRADING EN PYTHON

74 Trading System Performance

Trading Performance:

fundEquity

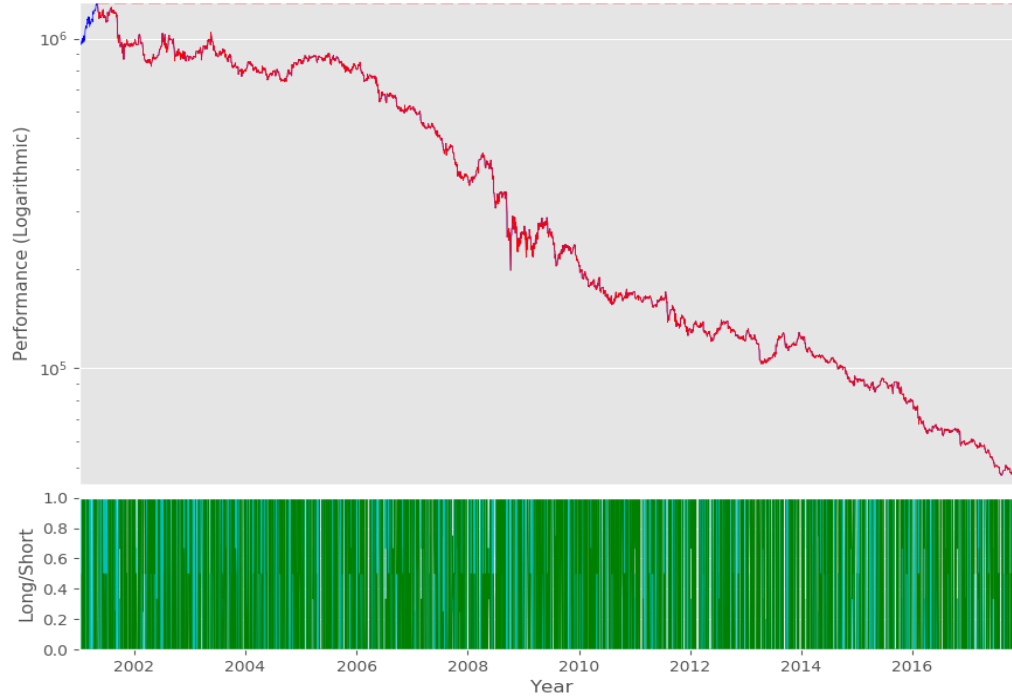
Exposure:

Long & Short

Market Returns:

BA

Trading Performance of fundEquity



Sharpe Ratio = -0.7024

Sortino Ratio = -1.0622

Performance (%/yr) = -0.1670

Volatility (%/yr) = 0.2378

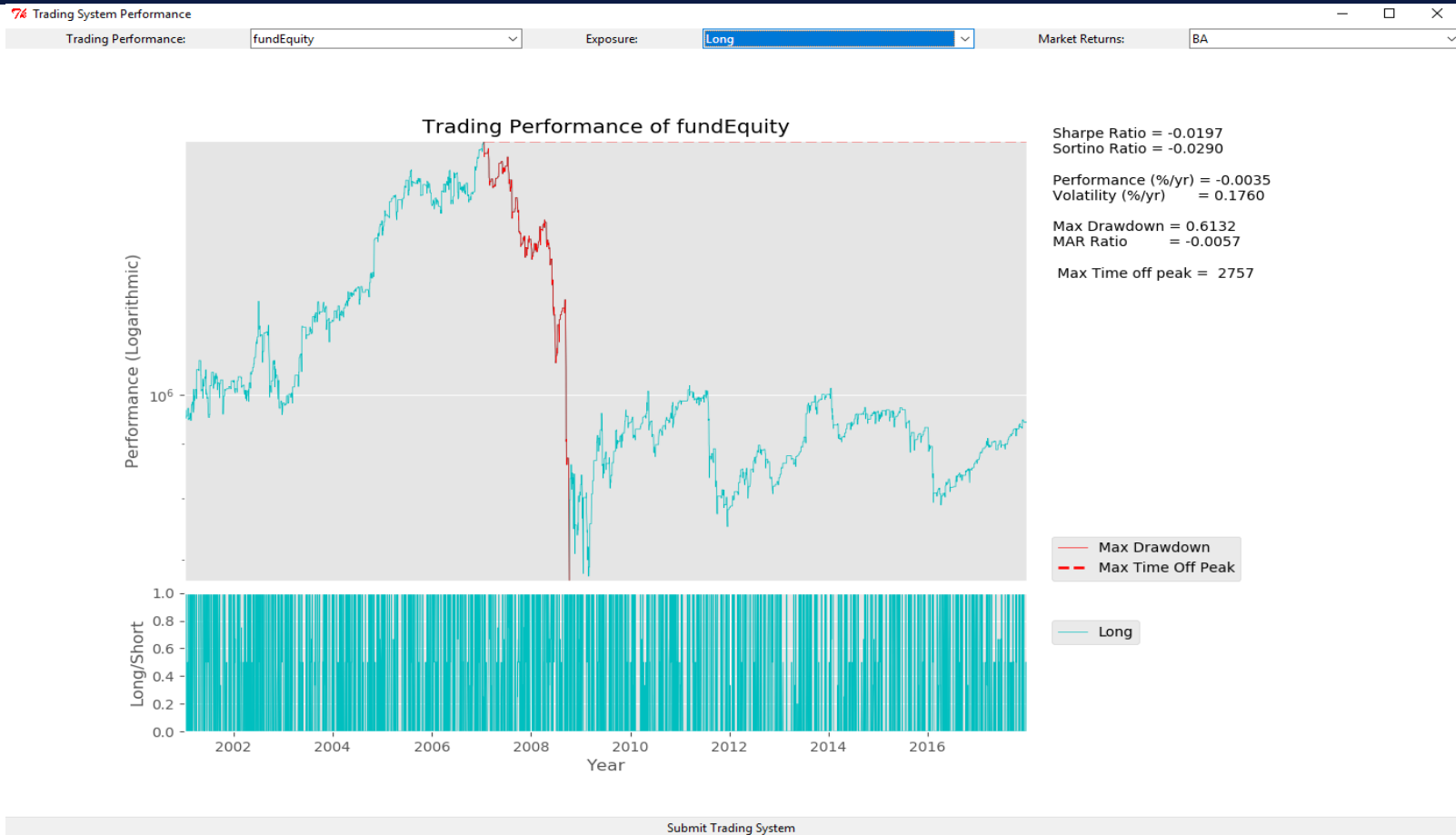
Max Drawdown = 0.9655

MAR Ratio = -0.1730

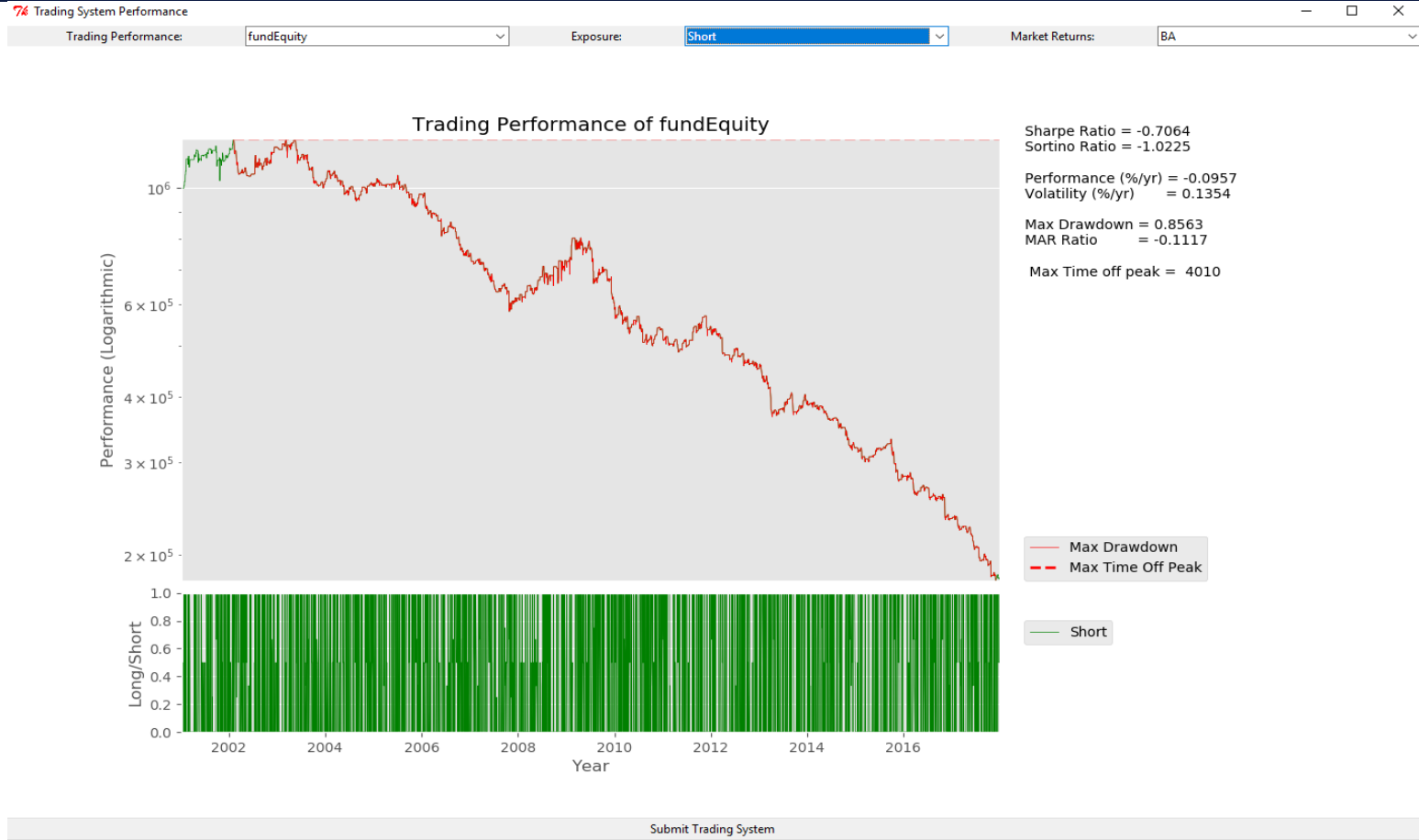
Max Time off peak = 4205

Submit Trading System

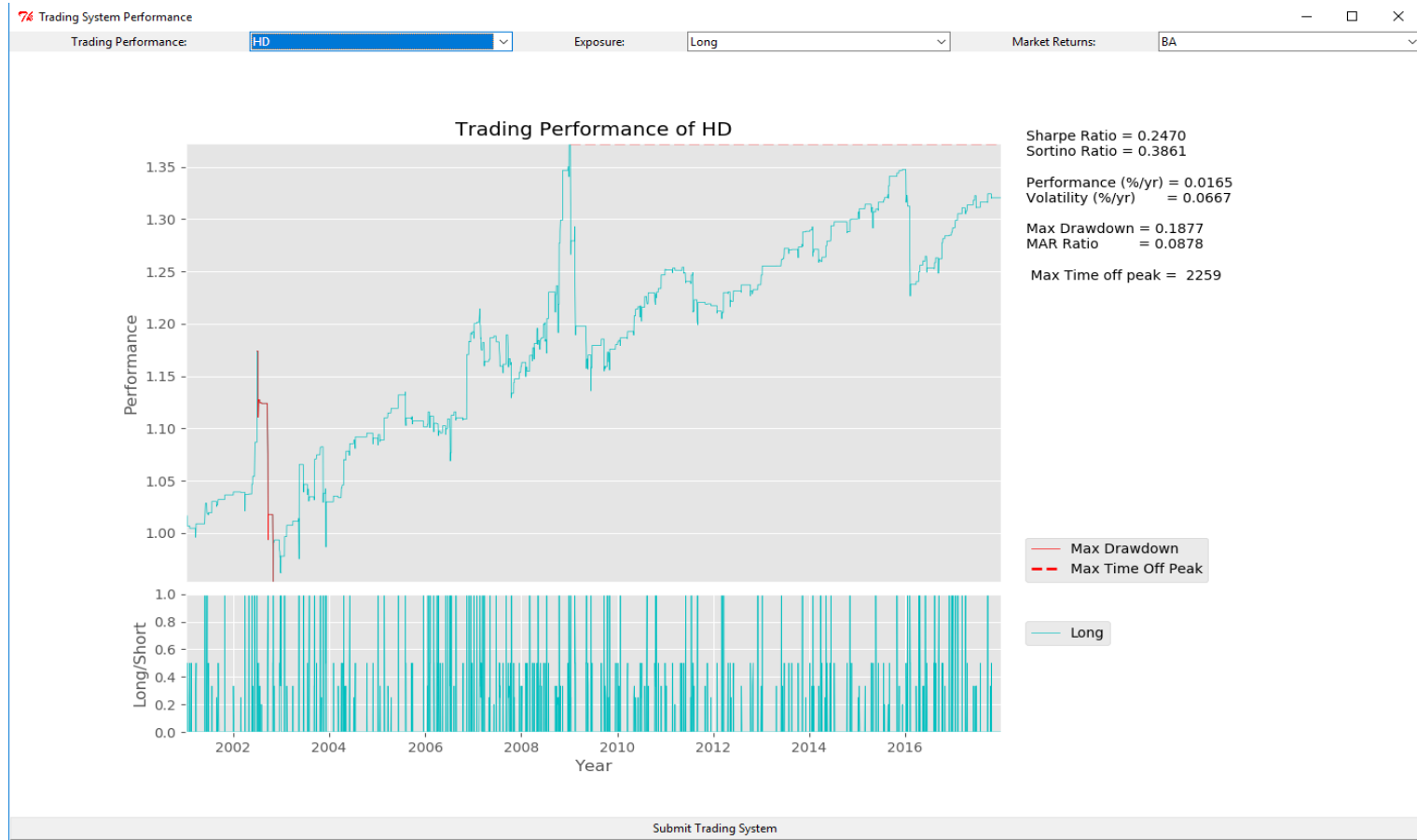
MI PRIMER SISTEMA DE TRADING EN PYTHON



MI PRIMER SISTEMA DE TRADING EN PYTHON



MI PRIMER SISTEMA DE TRADING EN PYTHON





MACHINE LEARNING EN PYTHON

MACHINE LEARNING EN PYTHON

iiiVamos a
Predecir
el Mercadoiii



MACHINE LEARNING EN PYTHON

Predecir el Mercado

Nuestro objetivo es muy simple:

- Si nuestra predicción es que Johnson & Johnson va a subir abrimos posición larga.
- Si nuestra predicción es que Johnson & Johnson va a bajar abrimos posición corta.

MACHINE LEARNING EN PYTHON

Predecir el Mercado

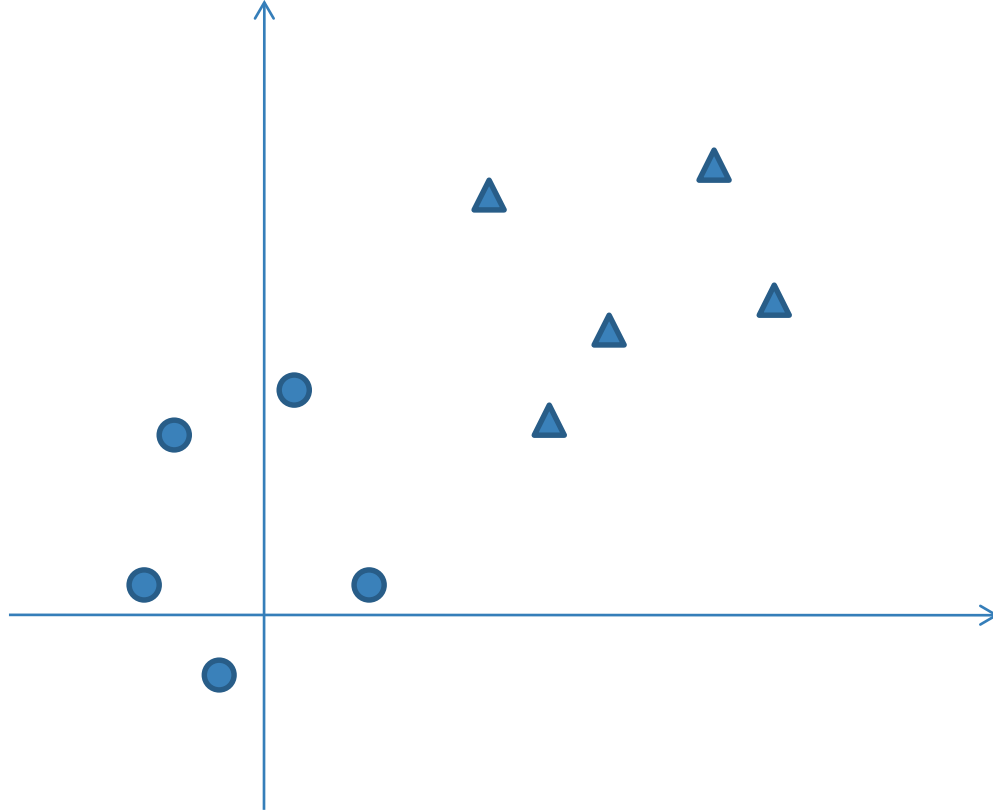
- Necesitaremos las siguientes librerías: Pandas, Numpy, Matplotlib, Seaborn y Scikit-Learn (contiene todos los algoritmos de machine learning)
- Como clasificador para este ejemplo utilizaremos el algoritmo de Support Vector Machines (SVM)

MACHINE LEARNING EN PYTHON

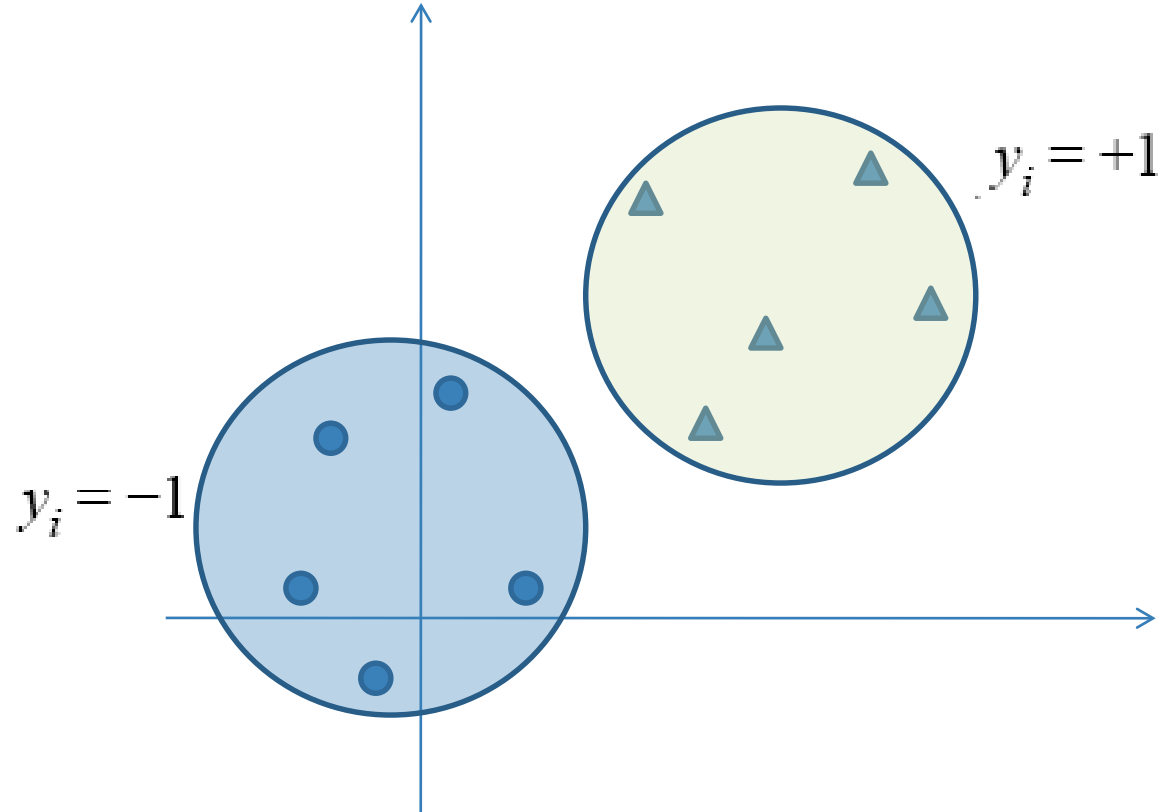
Support Vector Machines

- Desarrolladas por Vapnik y Cortés (1995), se trata de un conjunto de métodos de aprendizaje supervisados utilizados para la clasificación, regresión y detección de valores atípicos.
- El objetivo de SVM es encontrar el hiperplano que maximiza el margen entre dos o más clases. Los vectores (casos) que definen el hiperplano se denominan vectores soporte (*support vectors*).

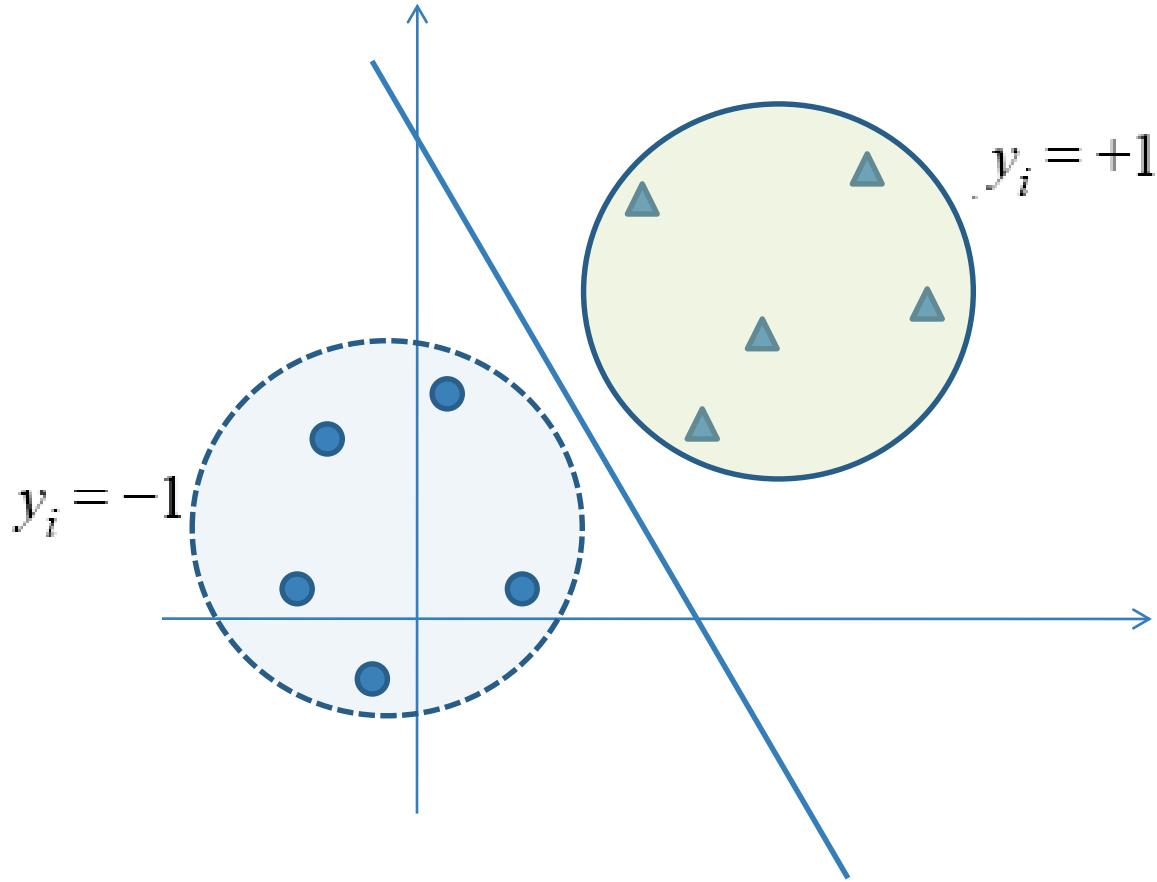
MACHINE LEARNING EN PYTHON



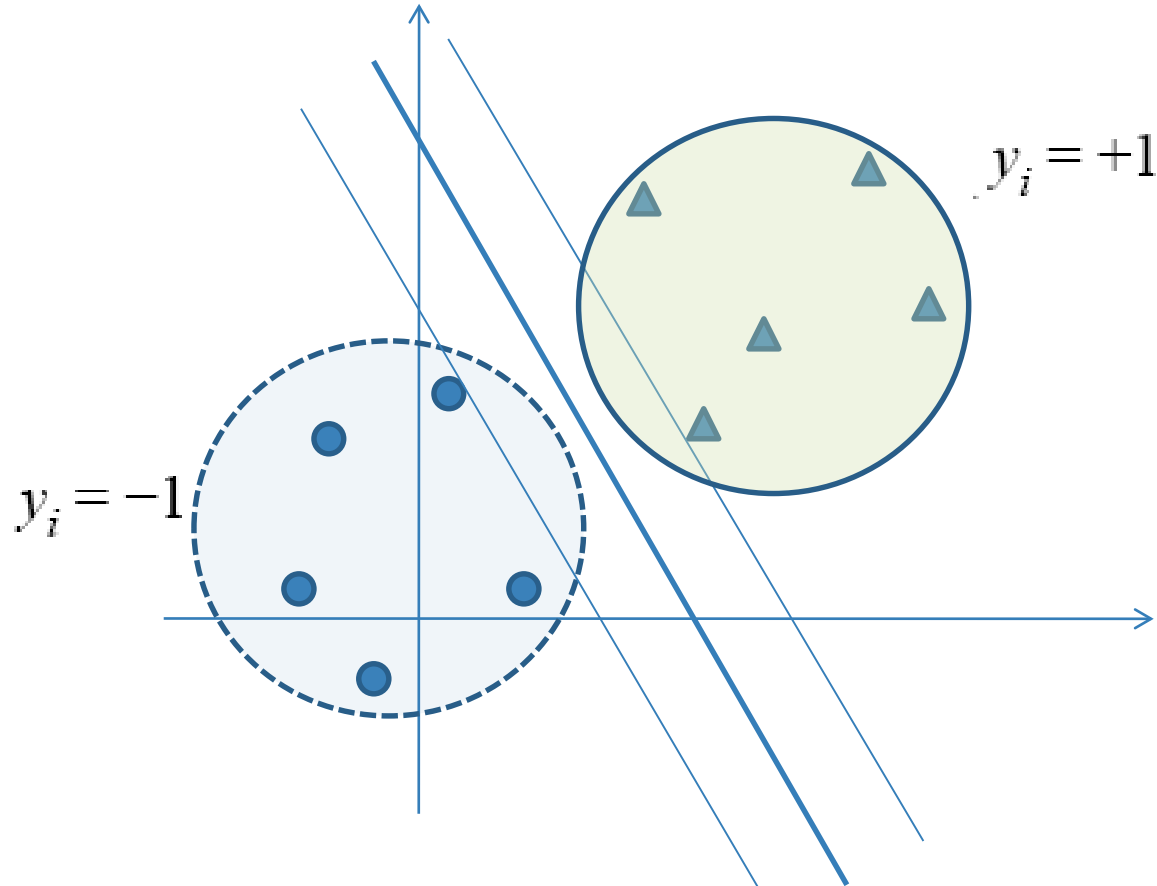
MACHINE LEARNING EN PYTHON



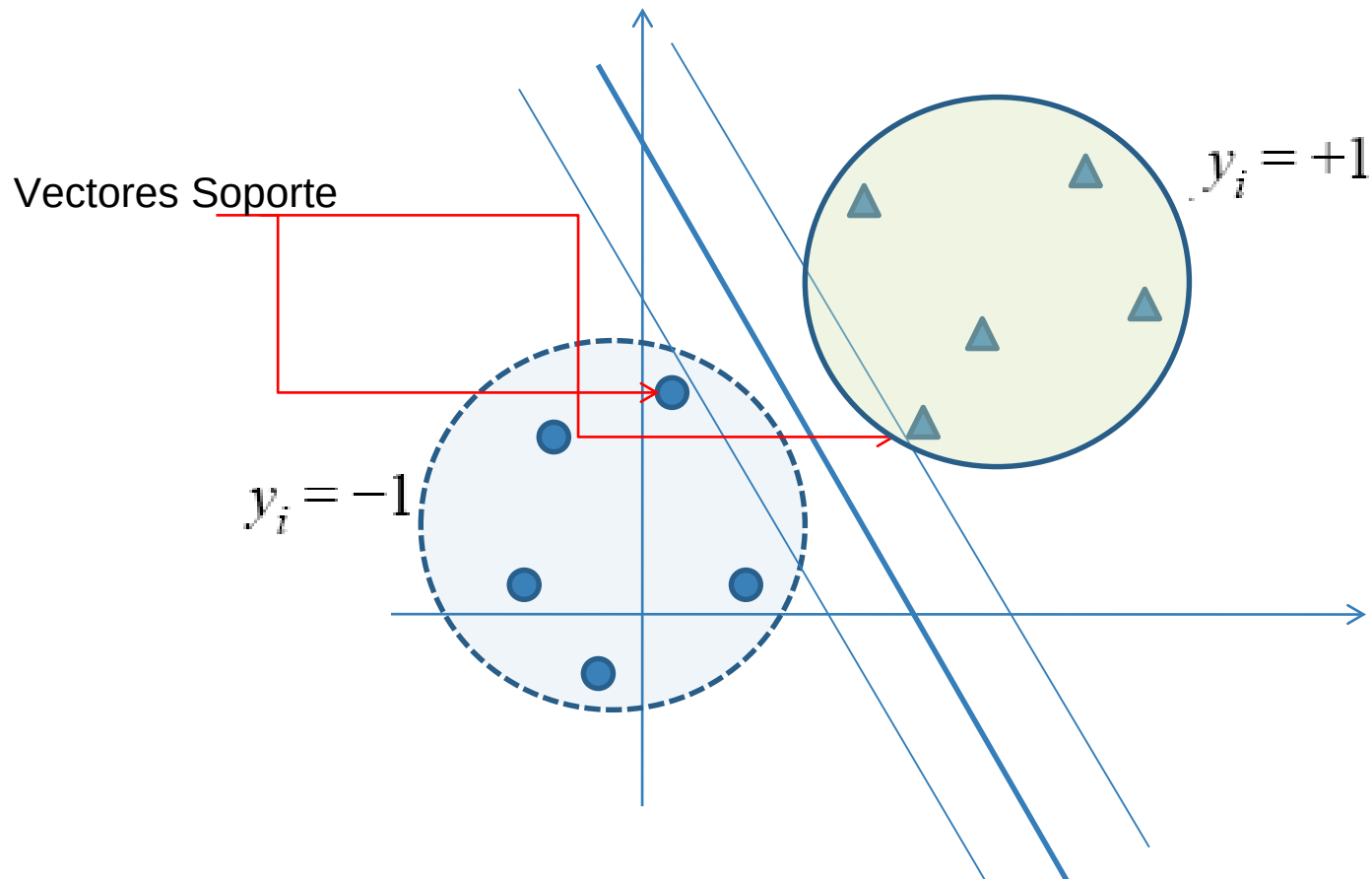
MACHINE LEARNING EN PYTHON



MACHINE LEARNING EN PYTHON



MACHINE LEARNING EN PYTHON



MACHINE LEARNING EN PYTHON

Support Vector Machines

Ventajas:

- Eficaz en espacios de grandes dimensiones.
- Cálculo eficiente.
- Versátiles (se puede personalizar el kernel).

Desventajas:

- No funcionan bien ante la presencia de mucho ruido.
- No proporciona estimaciones directas de probabilidad

MACHINE LEARNING EN PYTHON

Predecir el Mercado

Como predictores (X) vamos a usar los siguientes valores:

- Open – Close
- High – Low

Nuestro target (y) será saber si las acciones de JNJ suben (+1) o bajan (-1) al día siguiente.

MACHINE LEARNING EN PYTHON

Predecir el Mercado

Cargamos librerías, descargamos datos y graficamos serie:

```
import pandas as pd
import numpy as np
from sklearn.svm import SVC
from sklearn.metrics import scorer
from sklearn.metrics import accuracy_score

import matplotlib.pyplot as plt
import seaborn

import quandl
quandl.ApiConfig.api_key = 'WgQVD8Npfox1MN7SV3Nm'
```

MACHINE LEARNING EN PYTHON

Predecir el Mercado

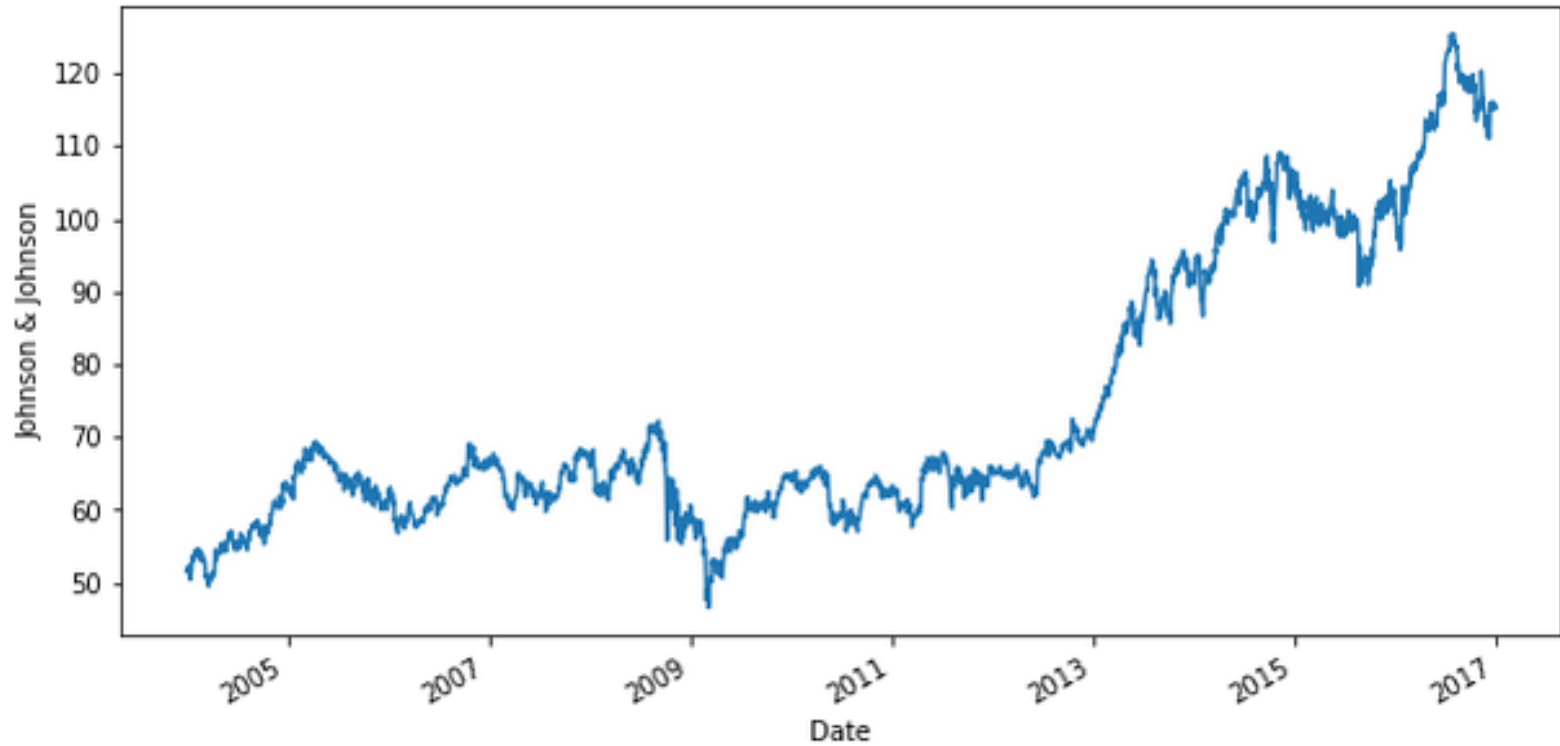
Cargamos librerías, descargamos datos y graficamos serie:

```
Df = quandl.get("WIKI/JNJ", start_date="2004-01-01", end_date="2016-12-31", returns="pandas")
```

```
Df= Df.dropna()
```

```
Df.Close.plot(figsize=(10,5))  
plt.ylabel("Johnson & Johnson")  
plt.show()
```

MACHINE LEARNING EN PYTHON



MACHINE LEARNING EN PYTHON

Predecir el Mercado

Definimos target y predictores:

```
y = np.where(Df['Close'].shift(-1) > Df['Close'],1,-1)
```

```
Df['Open-Close'] = Df.Open - Df.Close
```

```
Df['High-Low'] = Df.High - Df.Low
```

```
X=Df[['Open-Close', 'High-Low']]
```

MACHINE LEARNING EN PYTHON

Predecir el Mercado

Creamos conjuntos de entrenamiento y validación:

```
split_percentage = 0.8  
split = int(split_percentage*len(Df))
```

```
# Train data set  
X_train = X[:split]  
y_train = y[:split]
```

```
# Test data set  
X_test = X[split:]  
y_test = y[split:]
```

MACHINE LEARNING EN PYTHON

Predecir el Mercado

`sklearn.svm.SVC`

```
class sklearn.svm.SVC(C=1.0, kernel='rbf', degree=3, gamma='auto',  
coef0=0.0, shrinking=True, probability=False, tol=0.001, cache_size=200,  
class_weight=None, verbose=False, max_iter=-1,  
decision_function_shape='ovr', random_state=None)
```

MACHINE LEARNING EN PYTHON

Predecir el Mercado

Generamos SVM, lo entrenamos y validamos, obtenemos precisión:

```
cls = SVC().fit(X_train, y_train)

accuracy_train = accuracy_score(y_train, cls.predict(X_train))
accuracy_test = accuracy_score(y_test, cls.predict(X_test))

print('\nTrain Accuracy:{: .2f}%'.format(accuracy_train*100))
print('Test Accuracy:{: .2f}%'.format(accuracy_test*100))
```

Train Accuracy: 51.83%

Test Accuracy: 50.84%

MACHINE LEARNING EN PYTHON

Predecir el Mercado

Predecimos señal de compra con `cls.predict()`. Seguidamente calculamos rendimientos en base a las predicciones en el conjunto de validación. Finalmente graficamos rentabilidad acumulada:

```
Df['Predicted_Signal'] = cls.predict(X)
```

```
Df['Return'] = np.log(Df.Close.shift(-1) / Df.Close)*100
```

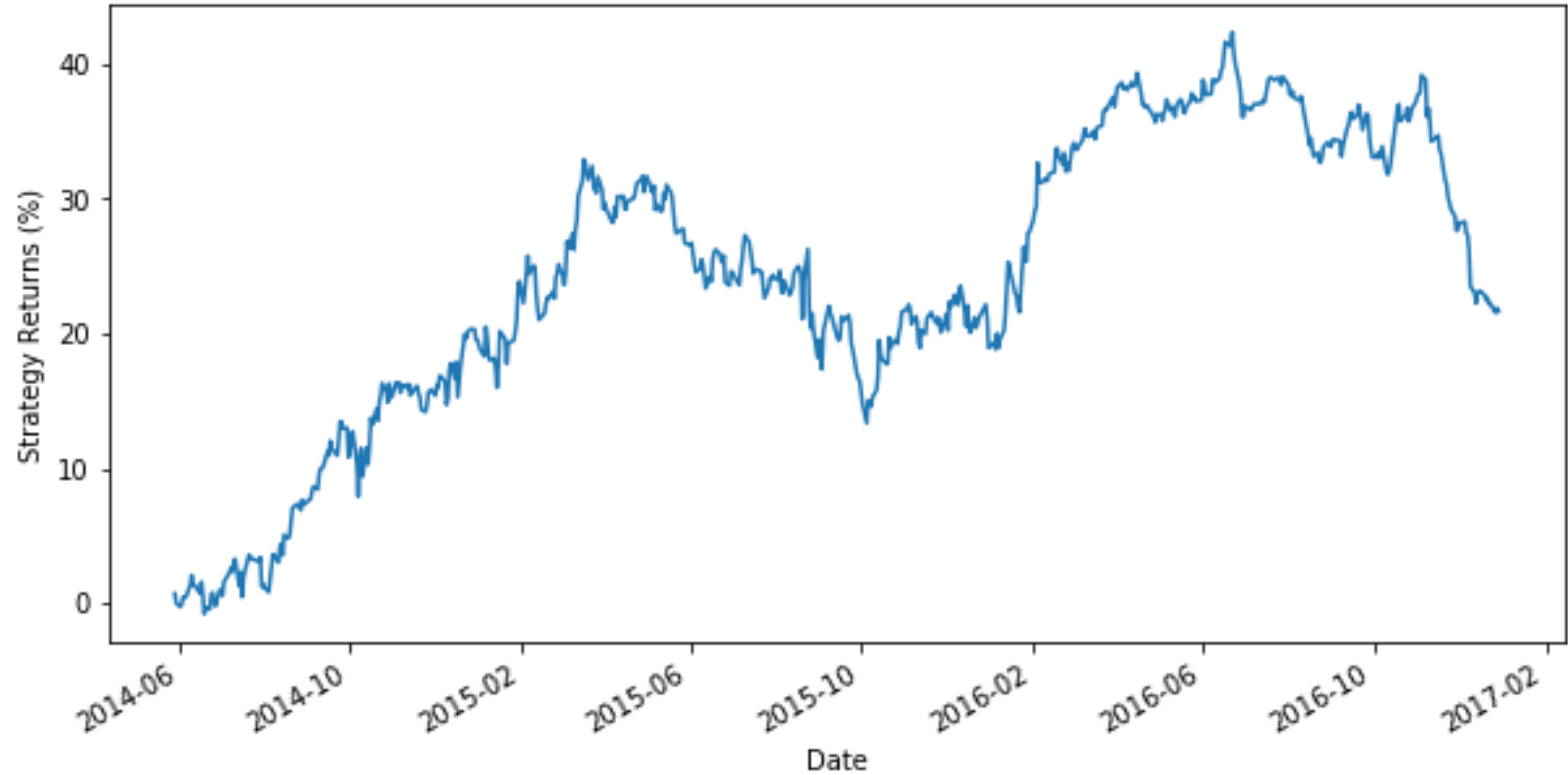
```
Df['Strategy_Return'] = Df.Return * Df.Predicted_Signal
```

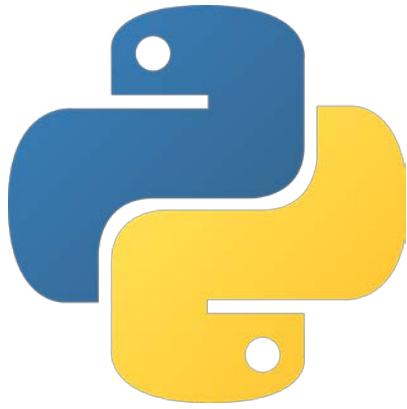
```
Df.Strategy_Return.iloc[split:].cumsum().plot(figsize=(10,5))
```

```
plt.ylabel("Strategy Returns (%)")
```

```
plt.show()
```


MACHINE LEARNING EN PYTHON

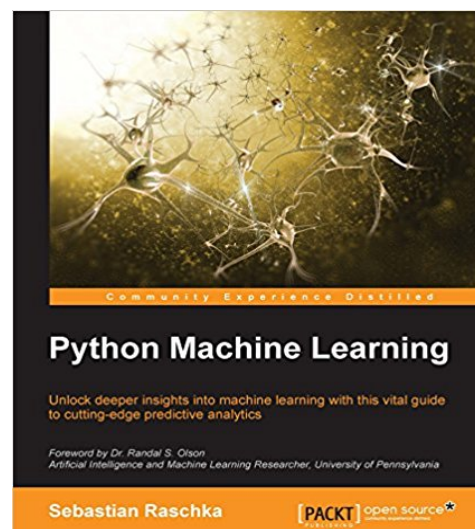
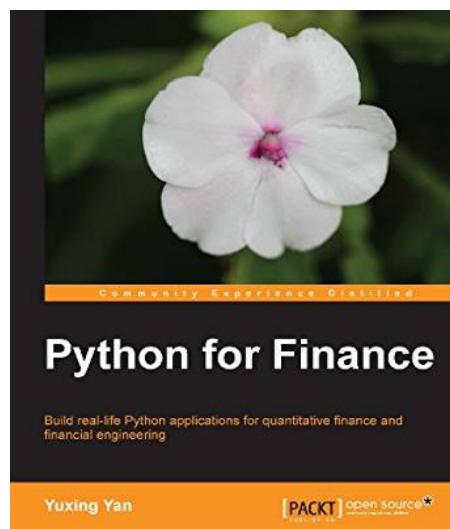
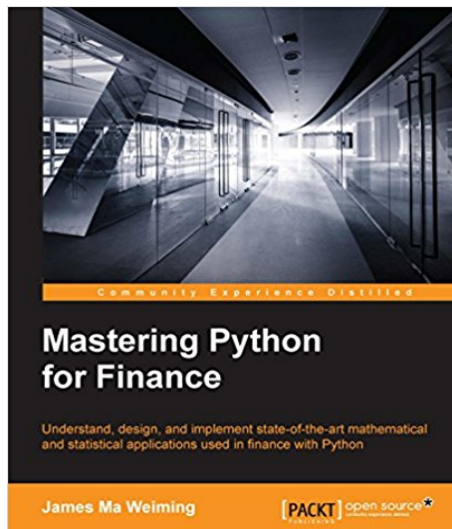




RECURSOS PARA TRADERS EN PYTHON

RECURSOS PARA TRADERS EN PYTHON

LIBROS



RECURSOS PARA TRADERS EN R

PÁGINAS WEB

Python for Finance: <http://www.pythonforfinance.net/>

Python Programming: <https://pythonprogramming.net/finance-tutorials/>

Trading with Python: <http://tradingwithpython.blogspot.com/>

Trading y Python: <https://tradingpython.blogspot.com.es/>

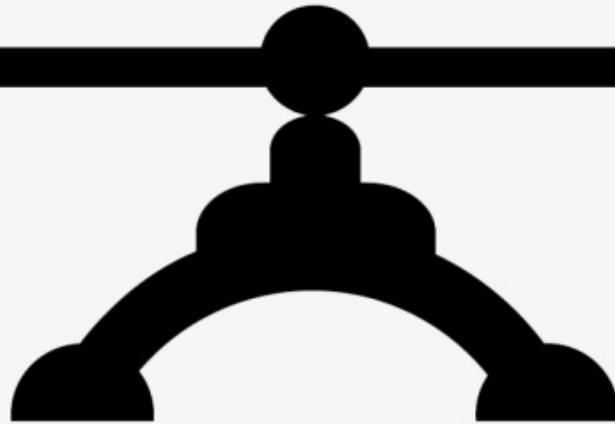
Futures.io: <https://futures.io/matlab-r-project-python/>

Quantopian: <https://www.quantopian.com/>

Quantiacs: <https://www.quantiacs.com/>

Numerai: <https://numer.ai/>

!!!EL VEREDICTO!!!

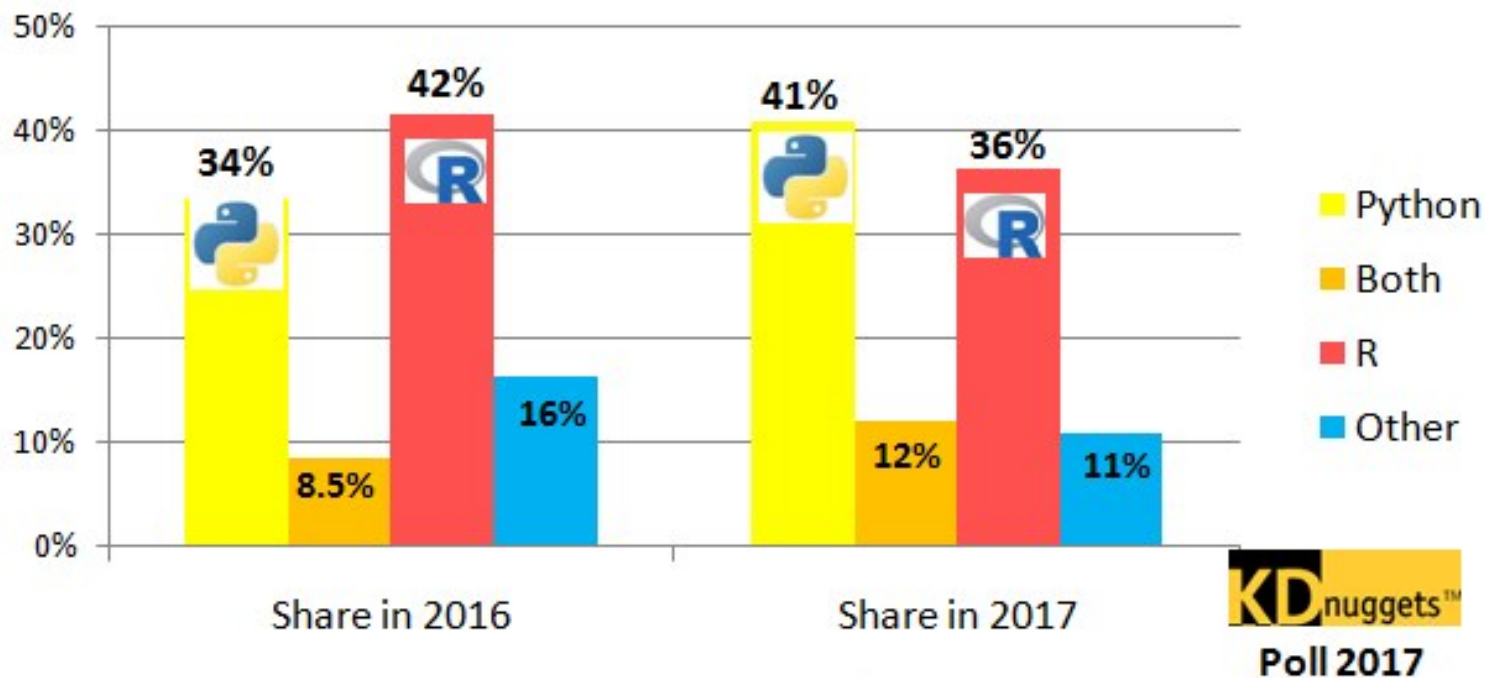


!!!EL VEREDICTO!!!

- Gráficos → *Ambos*
- Librerías → Por cantidad, **Python** // Por documentación, **R**
- Rapidez → *Ninguno*
- Sintaxis → **R**
- Un +1 para **Python** por Jupyter Notebook

!!!EL VEREDICTO!!!

Python, R, Both, or Other platforms for Analytics, Data Science, Machine Learning



www.x-trader.net



Volmageddon

Acaba de empezar 2018 y ya tenemos el primer crash, el denominado Volmageddon. Pero, ¿cuál ha sido la razón de las caídas del pasado 5 de febrero? En este artículo les ofrecemos todas las claves.



Mercado Forex. Predicción Basada en Montecarlo I



Carteras Core-Satellite Desde un Enfoque Sistemático

ARTÍCULOS DESTACADOS

Programa de Conferencias - Robotrader 2018

Algunas Funciones Poco Conocidas en TradeStation

Forks en Bitcoin

Fib Zone Pivots

Para Ganar, Primero Se Debe Sobrevivir

Usando los Osciladores al Revés

Usando Datos Propios en Build Alpha

Futuros sobre Bitcoin



Acuerdo entre Interactive Brokers y TradeStation



VINCHAIN ICO

Know your car's real history



JOIN ICO